Machine learning

Lecture 2

Tree-based Methods

Support Vector Machines

Inigo Bermejo

inigo.bermejo@uhasselt.be

DSI DATA SCIENCE INSTITUTE

UHASSELT

WWW.UHASSELT.BE/DSI

Tree-based Methods

Outline

- **Basics of decision trees**
  - How to build a tree
  - Tree pruning
  - Classification trees
  - Trees vs linear models
  - Advantages & disadvantages of trees
- Bagging
- Random forests
- Boosting

Basics of decision trees

Example

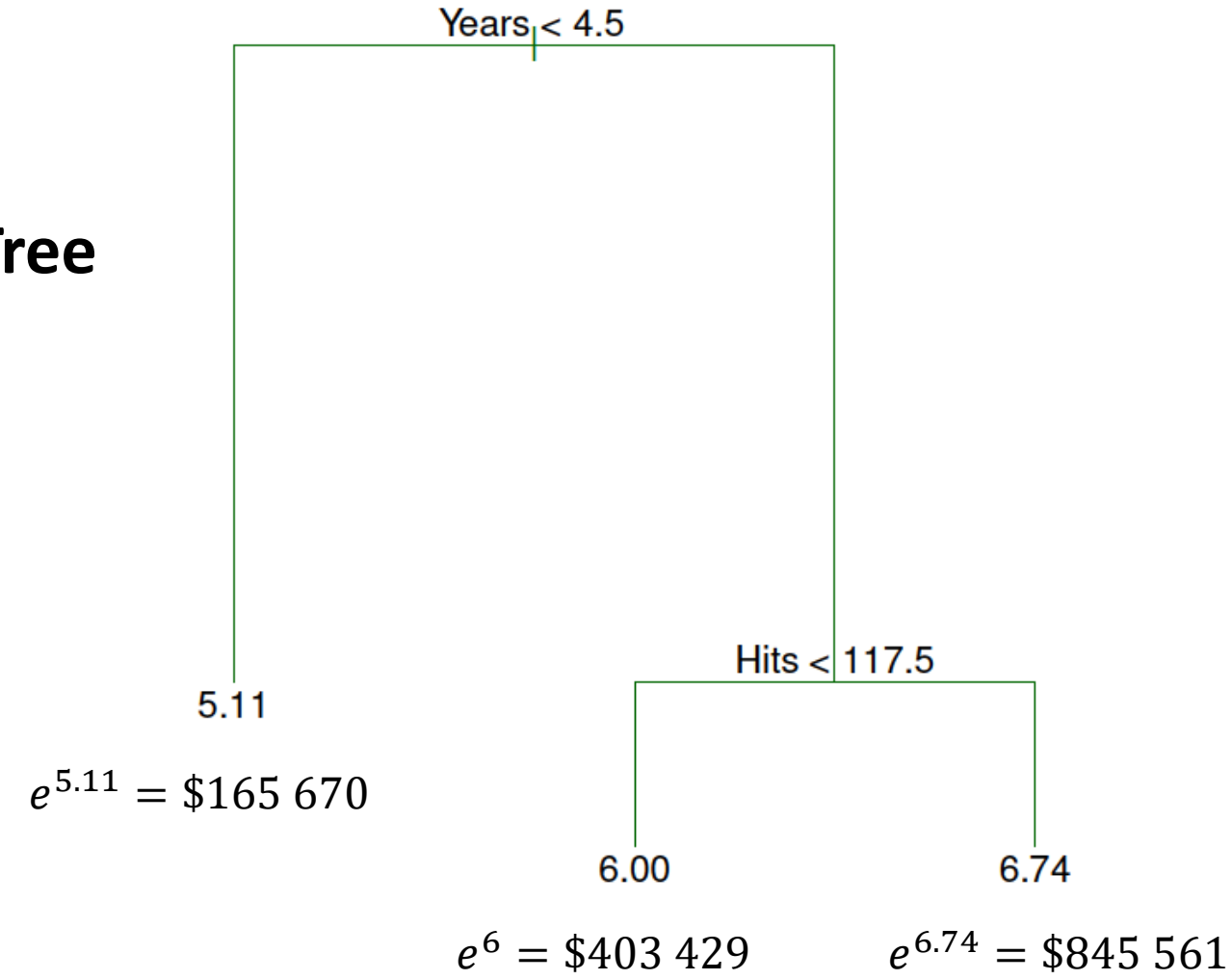
predict: $\ln(salary)$

from: $years\ played$ and $\#\ of\ hits$

Basics of decision trees

Example

Regression Tree



$e^{5.11} = \$165\ 670$

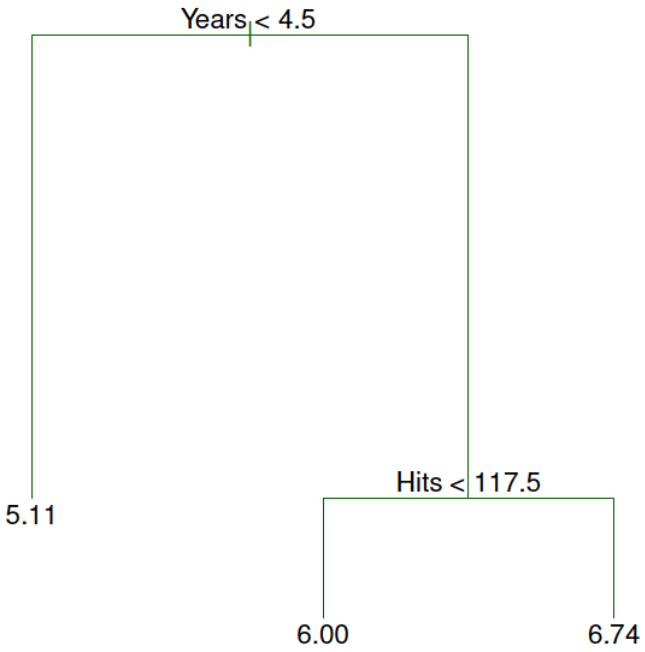
$e^{6} = \$403\ 429$

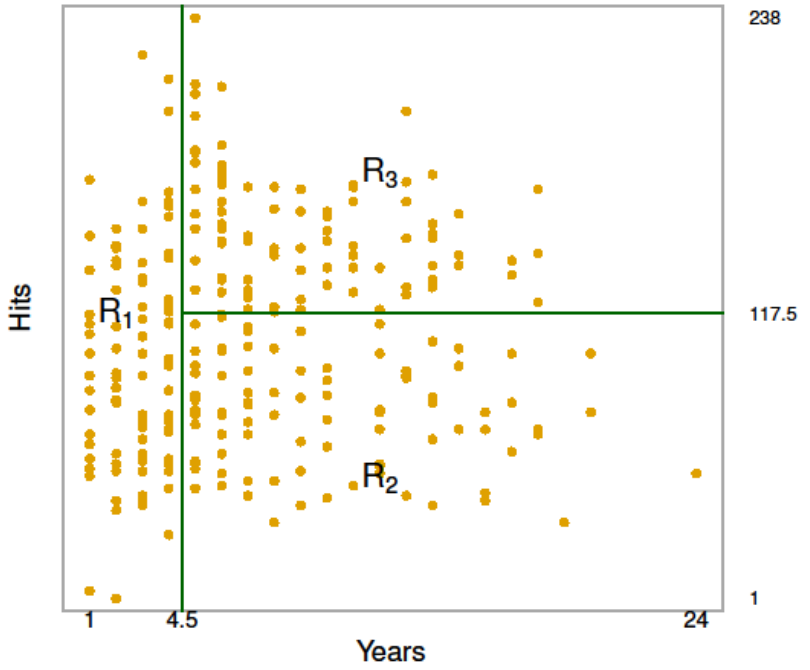
$e^{6.74} = \$845\ 561$

Basics of decision trees

Example

Space Partition





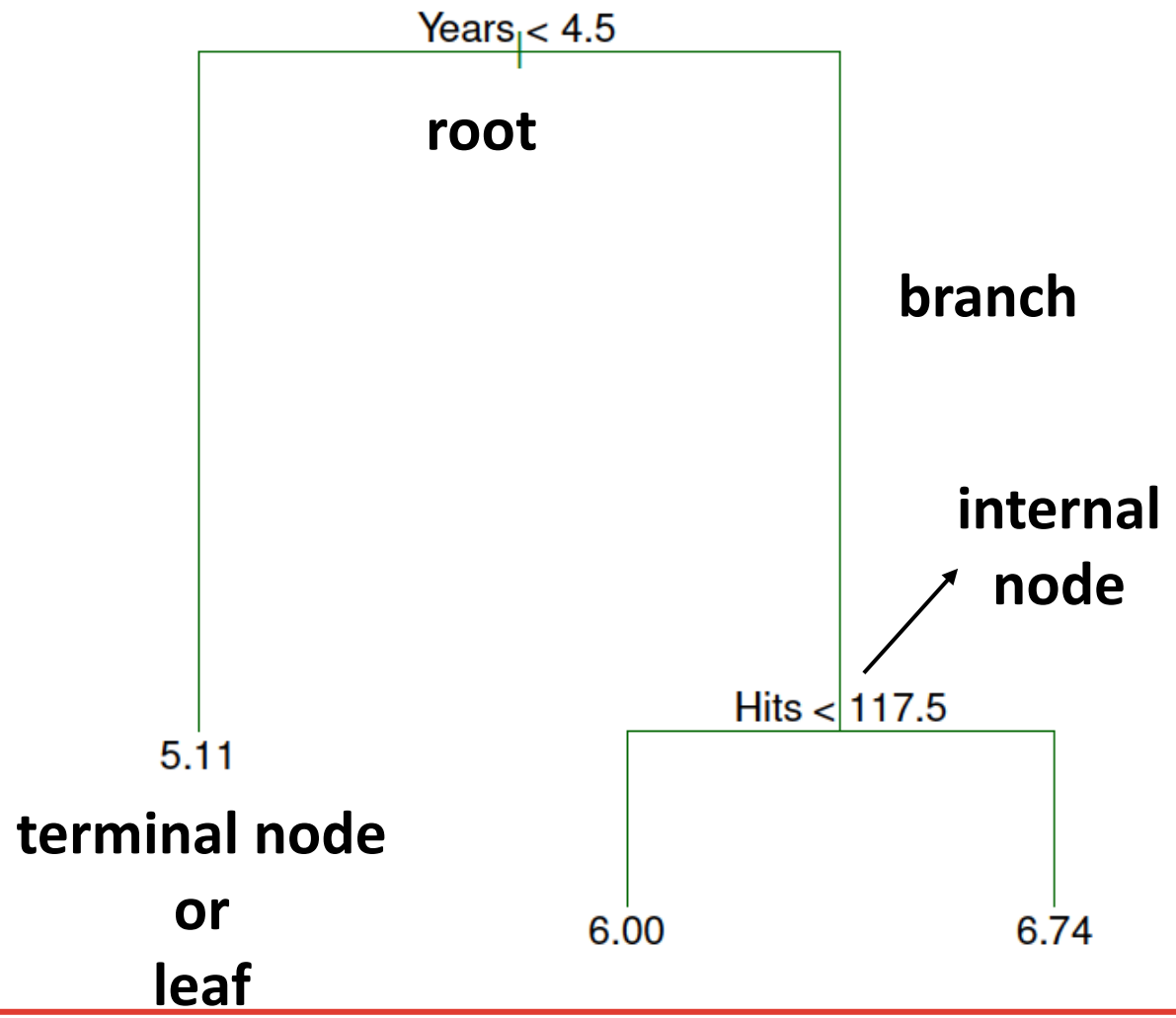
$$R_1 = \{X | Years < 4.5\}$$

$$R_2 = \{X | Years \geq 4.5, Hits < 117.5\}$$

$$R_3 = \{X | Years \geq 4.5, Hits \geq 117.5\}$$

Basics of decision trees

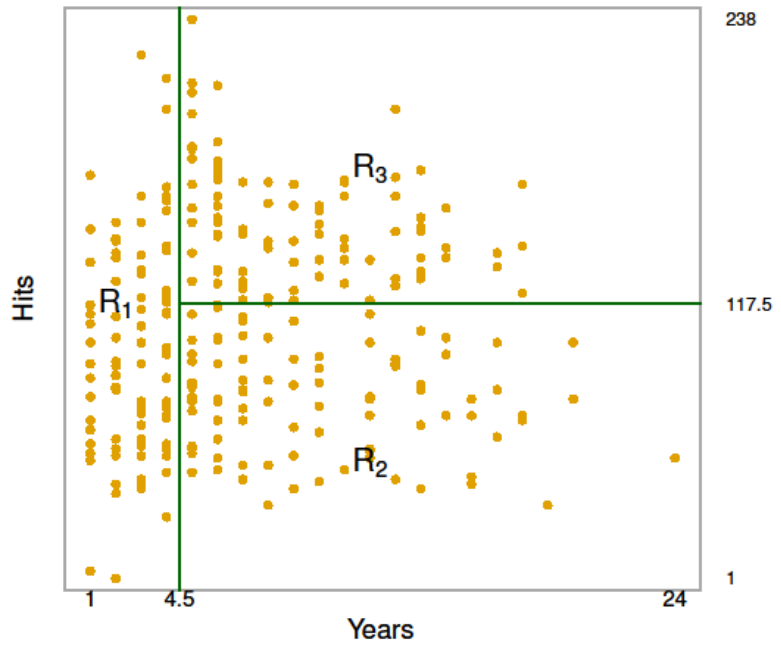
Terminology



How to build a tree

General process

1. Divide predictor space in non-overlapping regions $R_1, R_2, \ldots, R_J$
2. Every observation in region R_j = same prediction: mean / mode



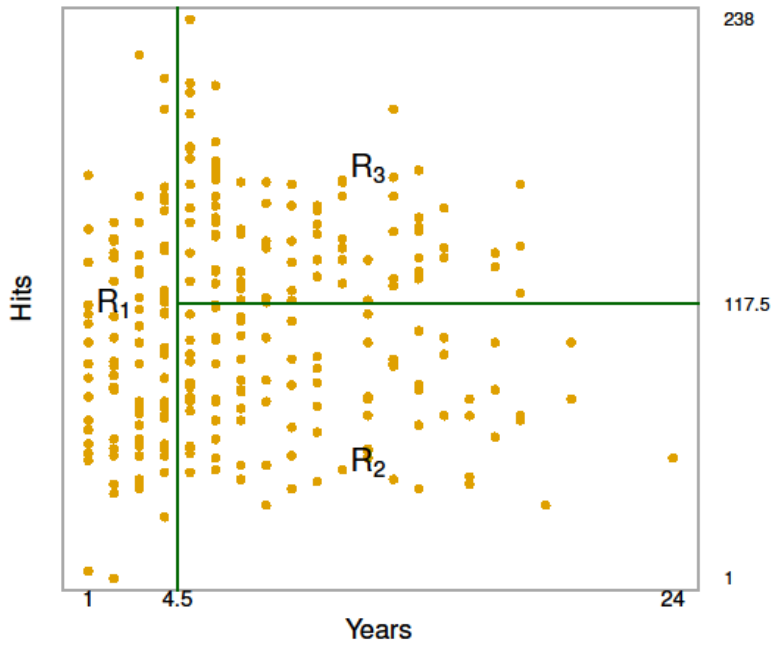
How to build a tree

Specific process

- Minimize $\text{RSS} = \sum_{j=1}^{J} \sum_{i \in R_j} \left(y_i - \hat{y}_{R_j} \right)^2$

Residual sum of squares

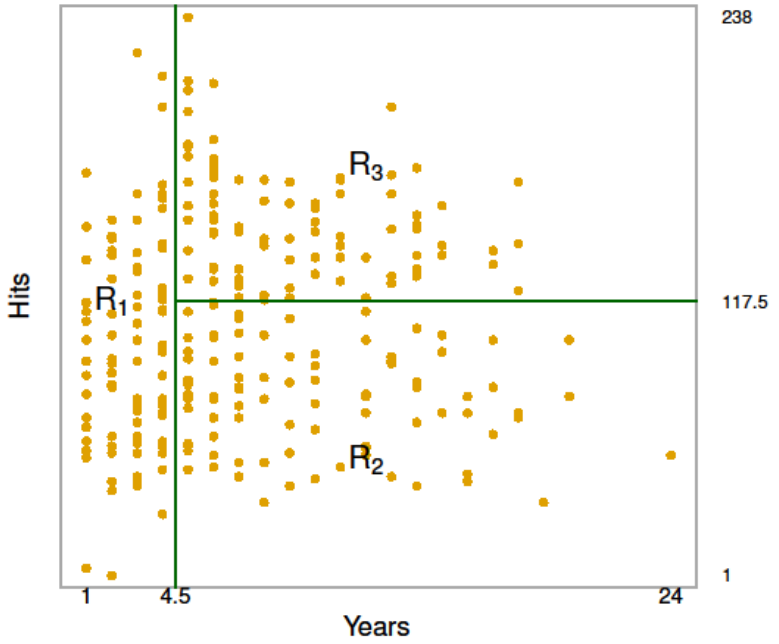
Average response in R_j



How to build a tree

Specific process

- High number of possible divisions
  - shape of regions
  - number of regions
- Finding optimal division is very complex



How to build a tree

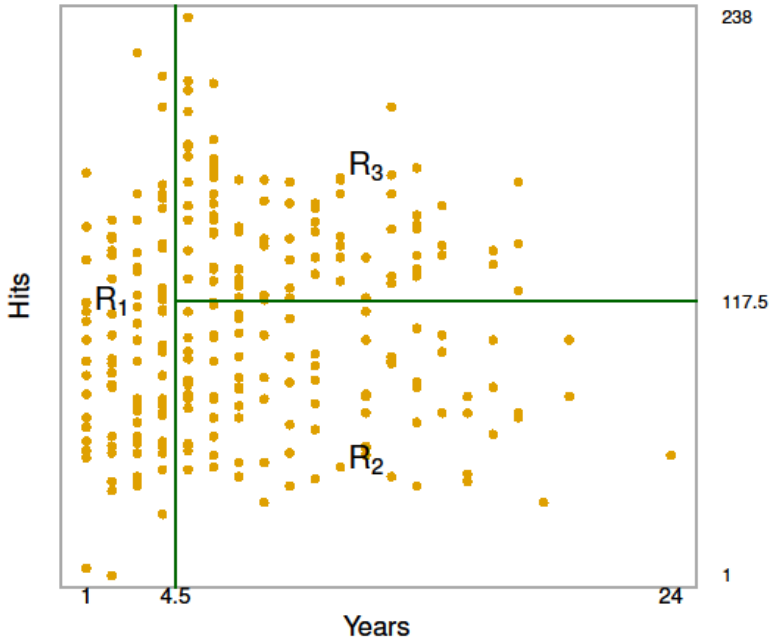
Specific process

We make some simplifying assumptions.

- Shape of regions: boxes (rectangles)
- Finding the optimal division:
  - Recursive Binary Splitting (RBS)
  - RBS is top-down and greedy

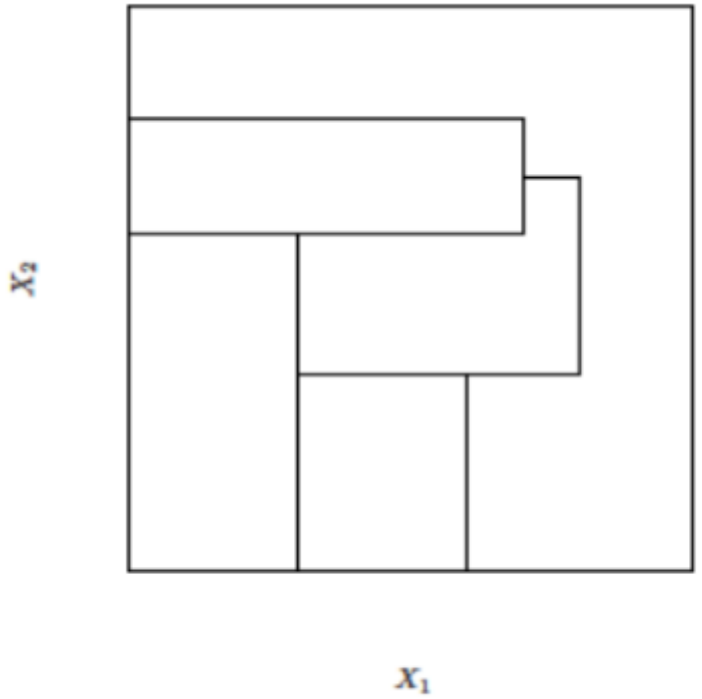
Starts at top of tree

Best split
One-step at a time
No looking ahead

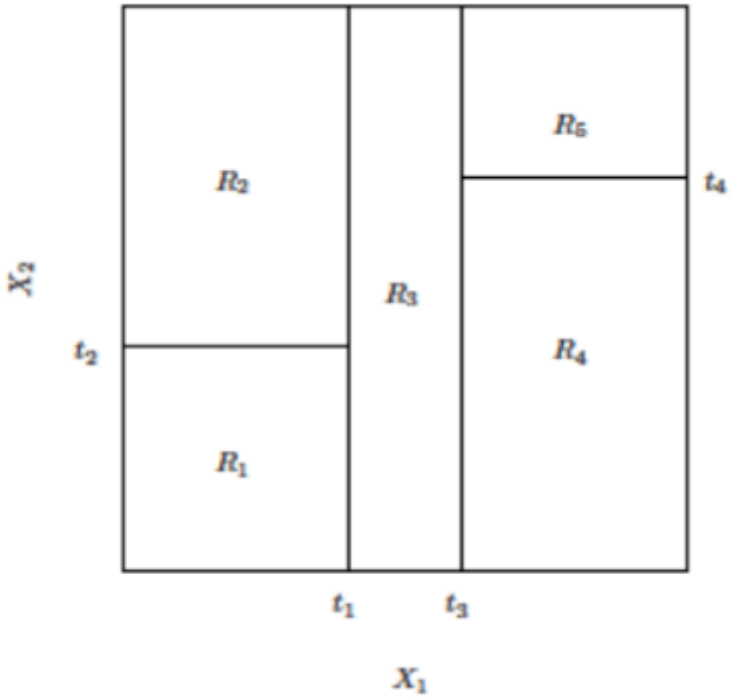


How to build a tree

Recursive Binary Splitting



Not possible with RBS



Result of RBS

How to build a tree

What is the "best" split?

- Consider:
  - all predictors $X_1, \ldots, X_p$
  - all cutpoints s
- And take split with lowest RSS.
- Mathematically, for all predictors j and any possible cutpoint s:

$$R_1(j,s) = \{X | X_j < s\} \text{ and } R_2(j,s) = \{X | X_j \geq s\}$$

- And take the minimum of:

$$\sum_{i:\, x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:\, x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2$$

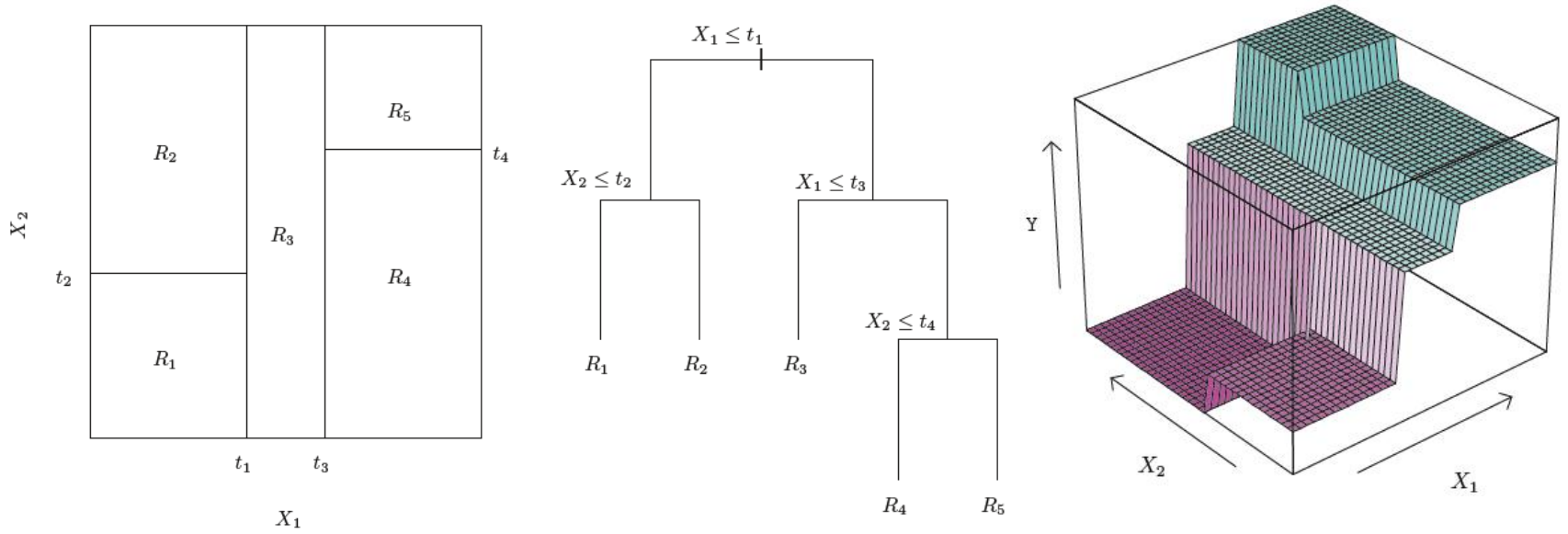
How to build a tree

What next?

Repeat same process again

- This time we don’t split the entire predictor space
- We split 1 of the 2 regions from previous step
- Now we have 3 regions
- Repeat until termination condition met
  - e.g. < 5 samples in any region

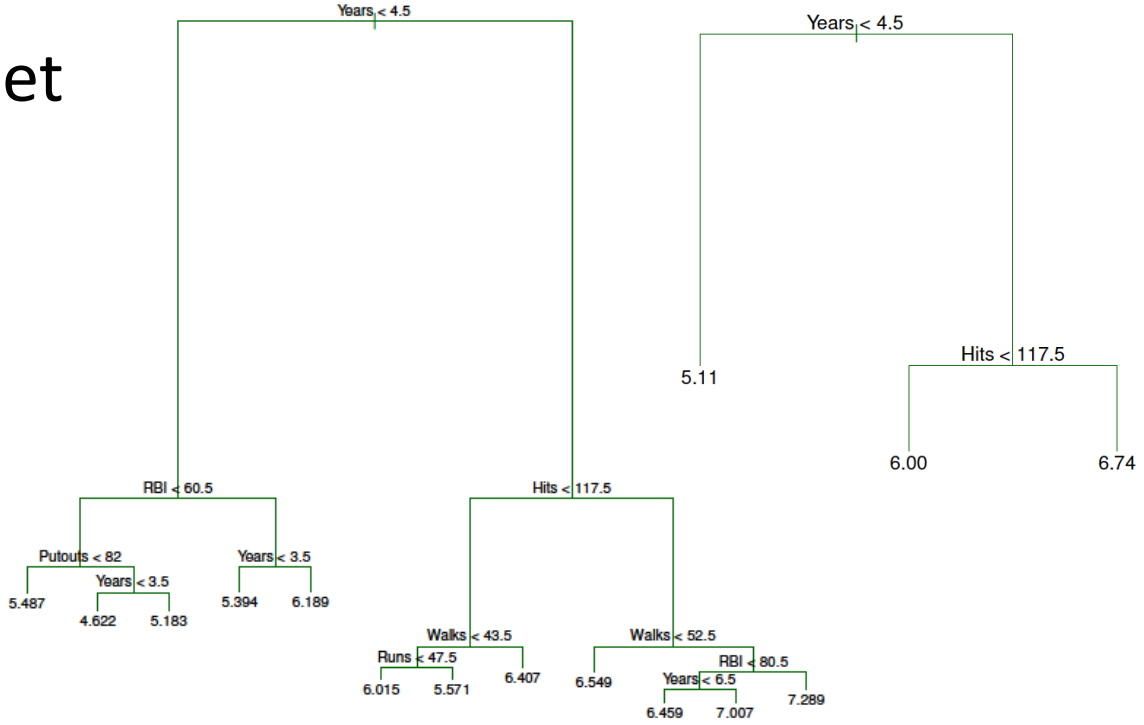
How to build a tree



Tree pruning

Problem statement

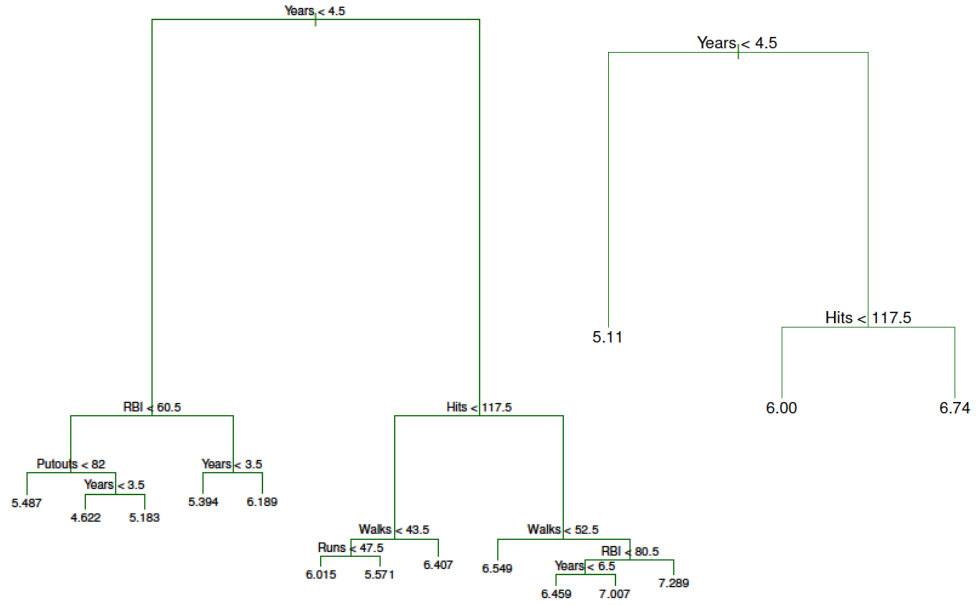
- Overfitting =
  model partly fits noise in training set
- In decision trees:
  too many splits ≈ overfitting
- Smaller tree →
  lower variance, higher bias



Tree pruning

Potential solution

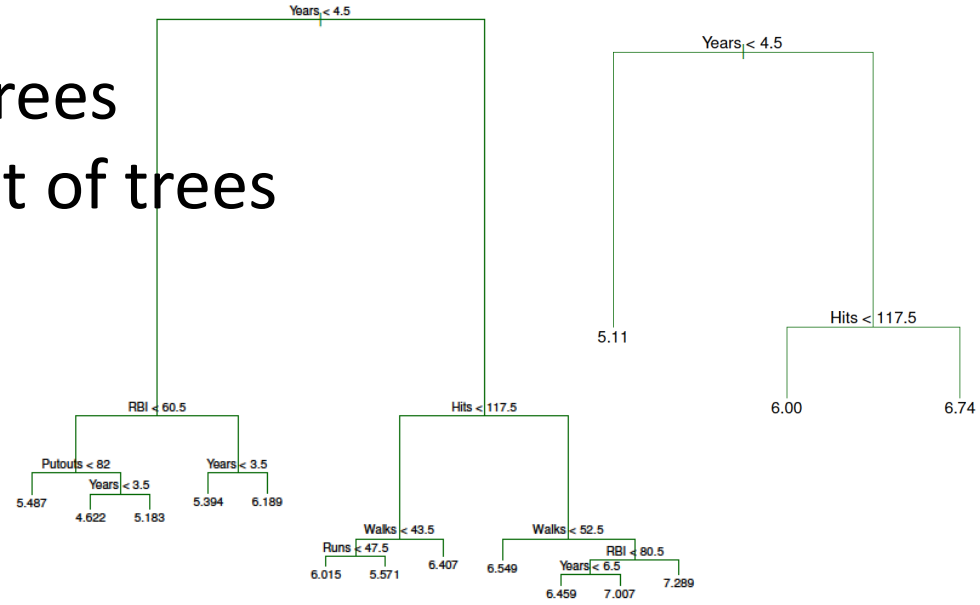
- Only split when decrease in RSS > threshold
- It turns out, this approach is too short-sighted
- Worthless split early on might lead to gains later on



Tree pruning

Alternative

- Grow large tree, prune it back
- How to best prune our tree?
  - Intuition: select subtree with lowest test RSS
  - We could use cross-validation
  - But there are too many possible sub-trees
  - We need a way to select a small subset of trees



Tree pruning

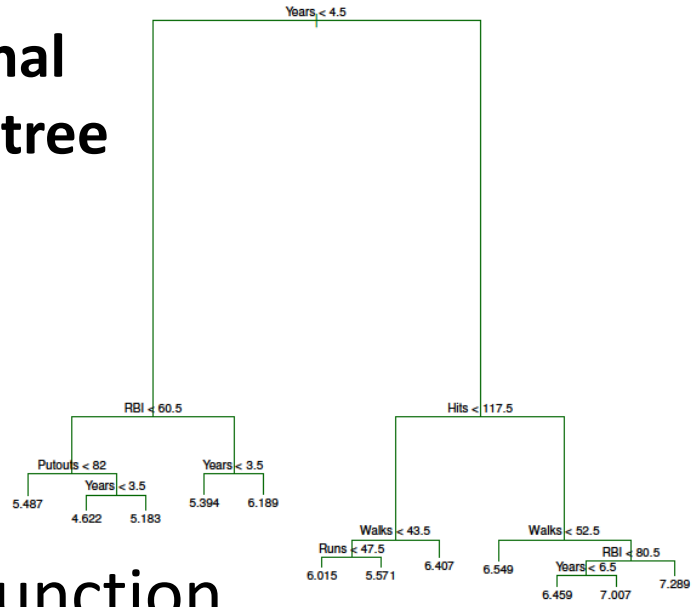
Cost complexity / weakest link pruning

- Add a regularization parameter α:

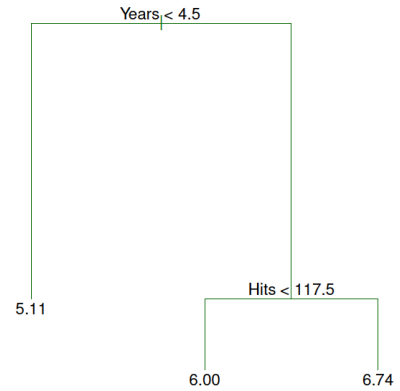
$$\sum_{m=1}^{|T|} \sum_{i:x_i \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2 + \alpha|T|$$

# of terminal nodes in subtree

RSS

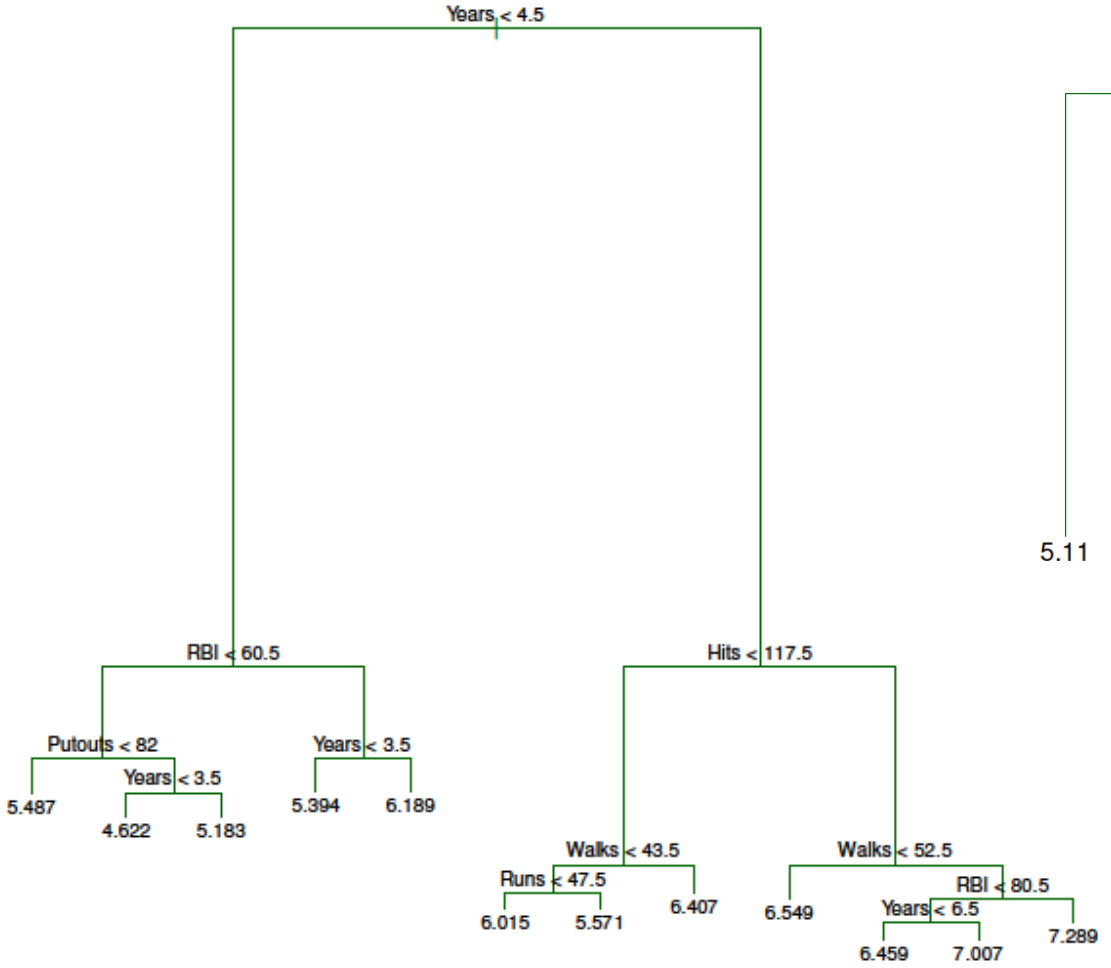


- For each α there is one subtree that minimises the function
- Trade-off between
  - low RSS
  - low $|T|$
- α is determined with cross-validation

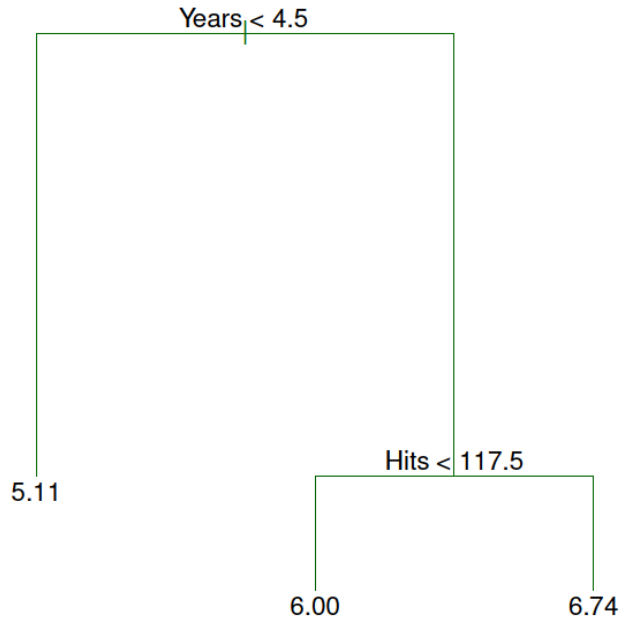


Tree pruning

Example



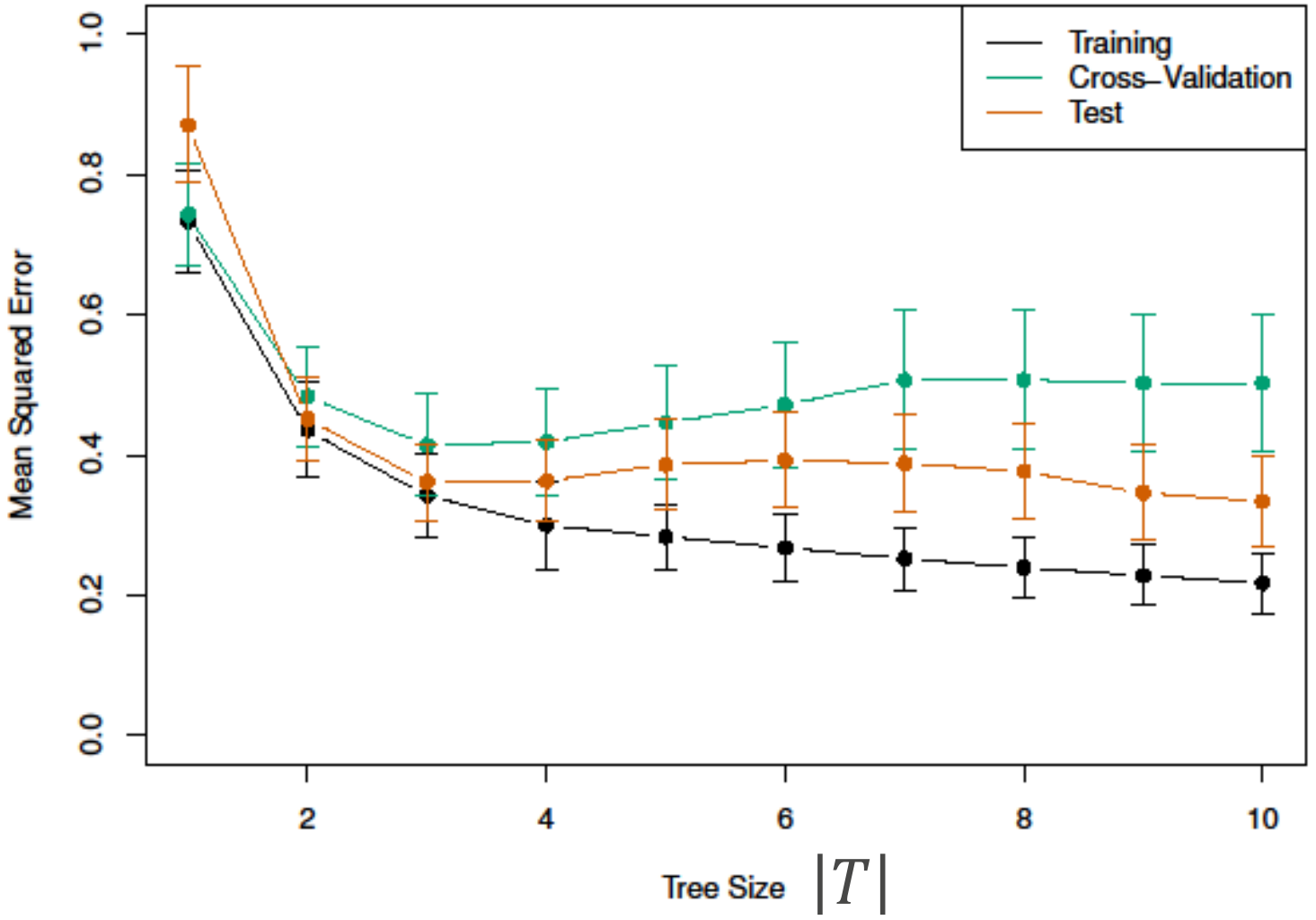
$|T| = 12$



$|T| = 3$

Tree pruning

Example



Classification trees

- Suppose we have qualitative classes
- Does a person have heart disease: Yes / No
- Prediction: most common class in region
- Classification error rate E instead of RSS

$$E = 1 - \max_k(\hat{p}_{mk})$$

% of class *k* in region *m*

Classification trees

Splitting metric

- Minimising E does not produce good splits
- Instead, we use:
  - Gini impurity

$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$$

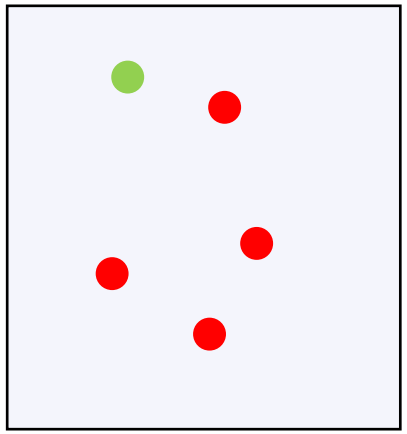
  - Entropy

$$D = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$$

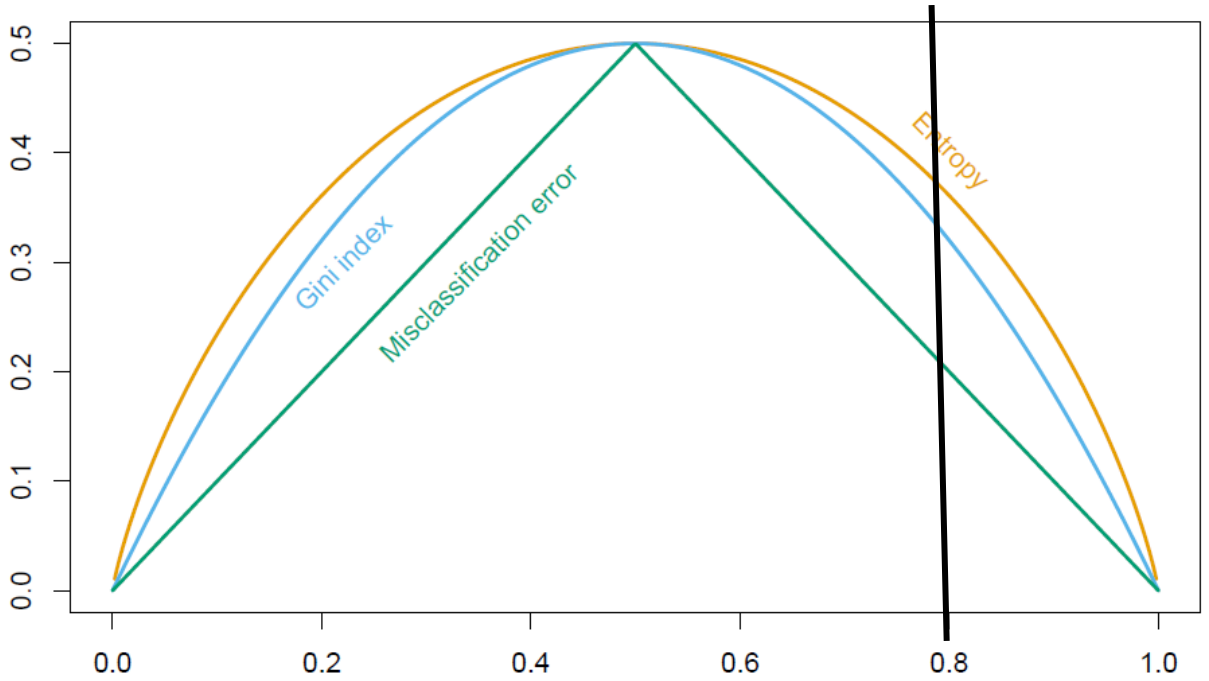
Classification trees

Impurity indices

class 0
class 1



R_m

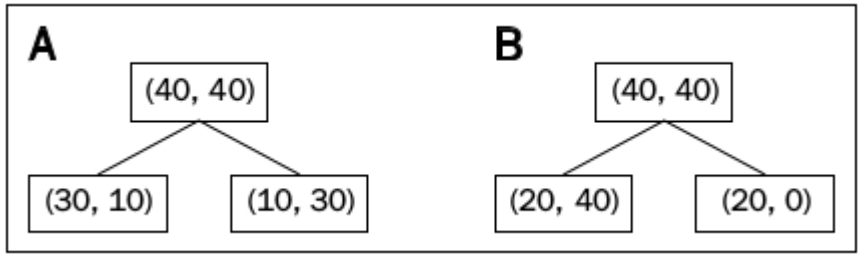


Gini index and Entropy favour *node purity.*

Classification trees

Impurity indices

Consider two splits:



$$A\colon\ E = \frac{\left(40 * \frac{10}{40} + 40 * \frac{10}{40}\right)}{80} = 1/4$$

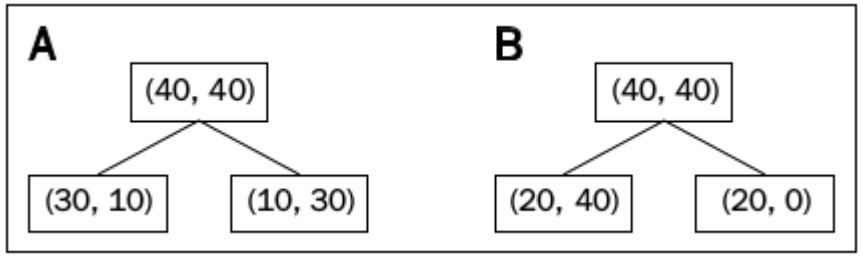
$$B\colon\ E = \frac{\left(60 * \frac{20}{60} + 20 * 0\right)}{80} = 1/4$$

Classification error has no preference.

Classification trees

Impurity indices

Consider two splits:



$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$$

$$A: G = \frac{\left(40 * 2 * \frac{10}{40} * (1 - \frac{10}{40}) + 40 * 2 * \frac{10}{40} * (1 - \frac{10}{40})\right)}{80} = \frac{6}{16} = 0.375$$

$$B: G = \frac{\left(60 * 2 * \frac{10}{30} * (1 - \frac{10}{30}) + 20 * 0\right)}{80} = \frac{1}{3} = 0.333 \ldots$$

Gini index and Entropy are more sensitive to node purity.

Classification trees

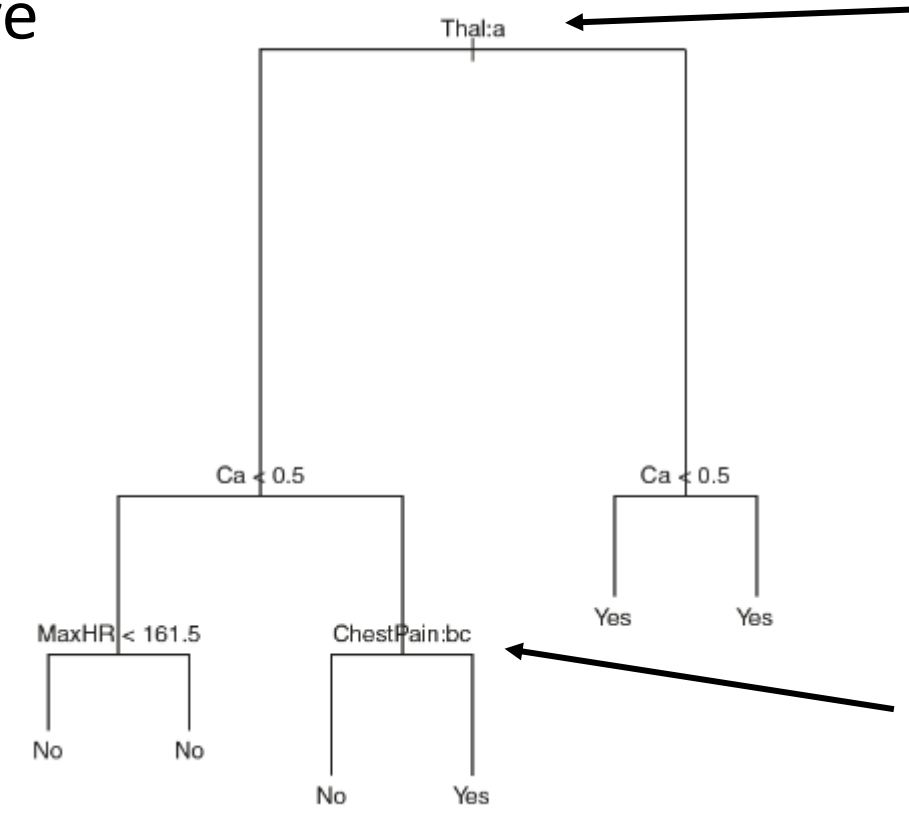
Remarks

- Build tree with Gini index / Entropy
- Prune tree with Error rate / Gini index / Entropy
  - If prediction accuracy is goal, error rate preferred

Classification trees

Remarks

- Trees handle qualitative variables naturally



Thallium stress test:

a. normal (left)
b. fixed (right)
c. reversible defects (right)

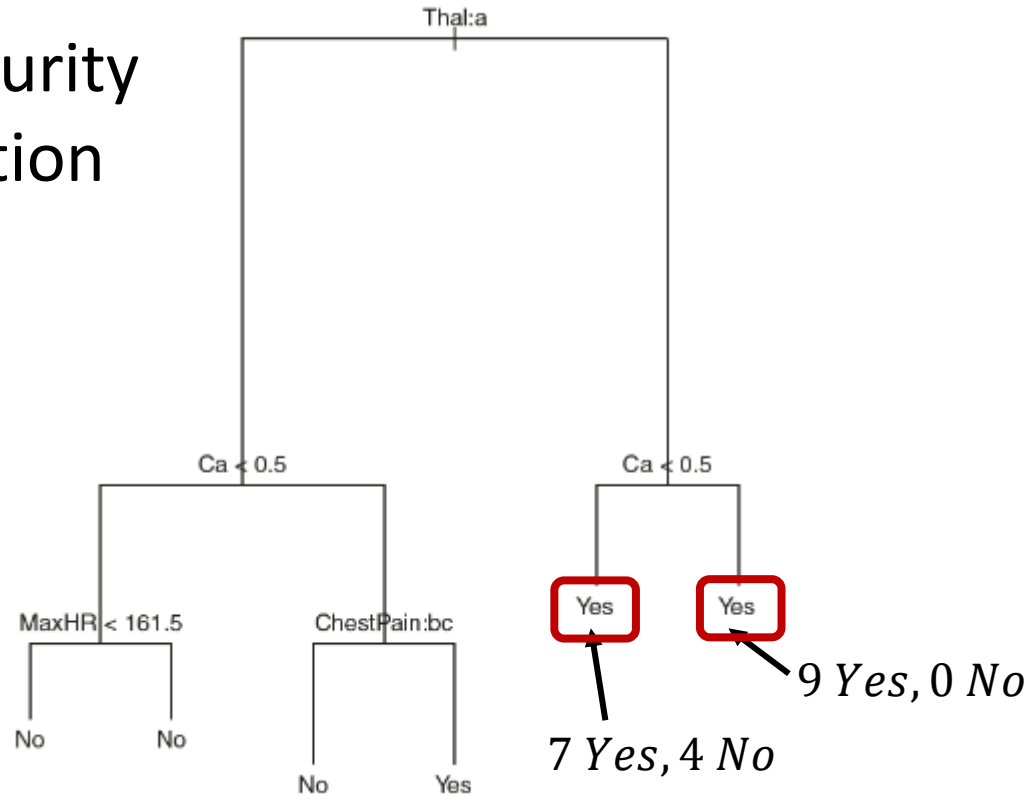
Chest pain:

a. typical angina (right)
b. atypical angina (left)
c. non-anginal pain (left)
d. asymptomatic (right)

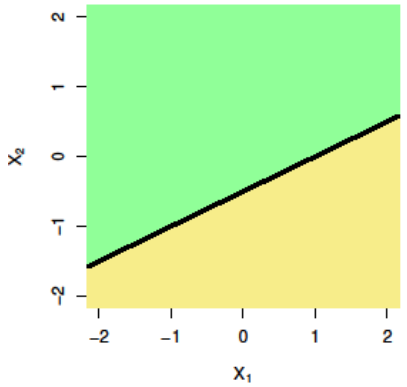
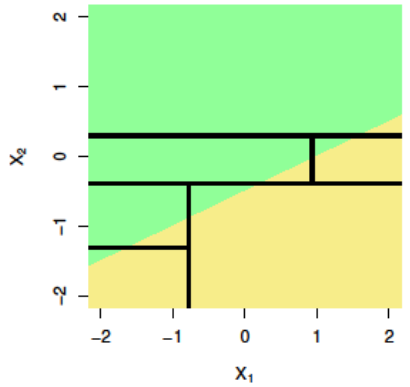
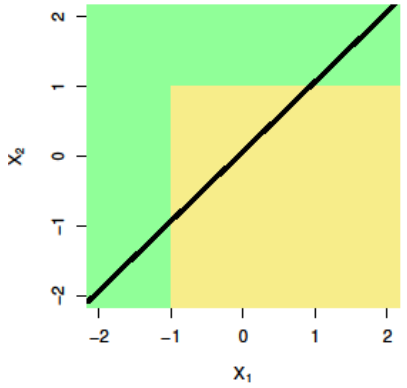
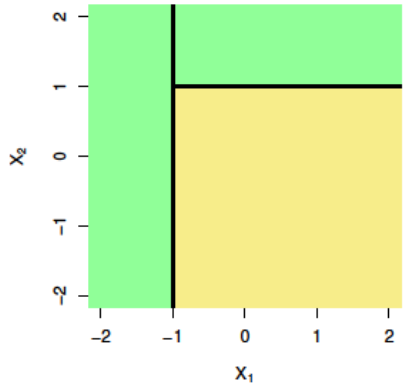
Classification trees

Remarks

- Some splits produce the same prediction
- This is still informative because of node purity
- Purity determines the certainty of prediction



Trees Versus Linear Models

	Linear model	Tree
	$f(X) = \beta_0 + \sum_{j=1}^{p} X_j \beta_j$	$f(X) = \sum_{m=1}^{M} c_m \cdot 1_{(X \in R_m)}$
Linear decision boundary		
Non-linear decision boundary		

Advantages & disadvantages of trees

- ✓ Easy to explain
- ✓ More closely mirror human decision-making (?)
- ✓ Can be displayed graphically
- ✓ Can handle qualitative predictors
- • Not as accurate as other regression & classification approaches
- • Can be very non-robust

Bagging and boosting can help!

Outline

- Basics of decision trees
  - How to build a tree
  - Tree pruning
  - Classification trees
  - Trees vs linear models
  - Advantages & disadvantages of trees
- **Bagging**
- Random forests
- Boosting

Bagging

Motivation

- Decision trees suffer from high variance.
- Slight variations in the training data produce highly variable trees.
- That is: trees are not very robust to noise.

Bagging

Naïve solution

- Create (or collect) $b = 1, 2, \ldots, B$ different training sets
- Train a classifier $\hat{f}^b(x)$ on each training set
- The average of these classifiers will have a lower variance:

$$\hat{f}_{\text{avg}}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^b(x)$$

But collecting many training sets is difficult / expensive!

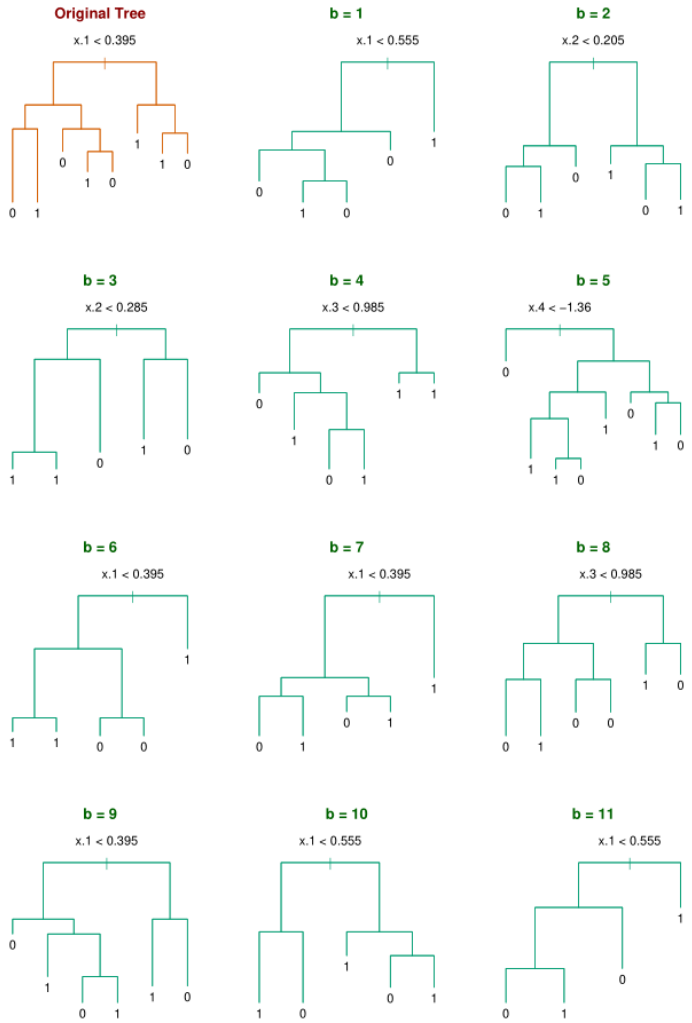
Bagging

- Instead, we simulate many datasets: *bootstrap aggregation / bagging*
- Create $b = 1, 2, \ldots, B$ bootstraps of training set:
- Train a classifier $\hat{f}^{*b}(x)$ on each bootstrap
- Take the average prediction:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^{*b}(x)$$

Majority voting for classification

Bagging



Source: ESL

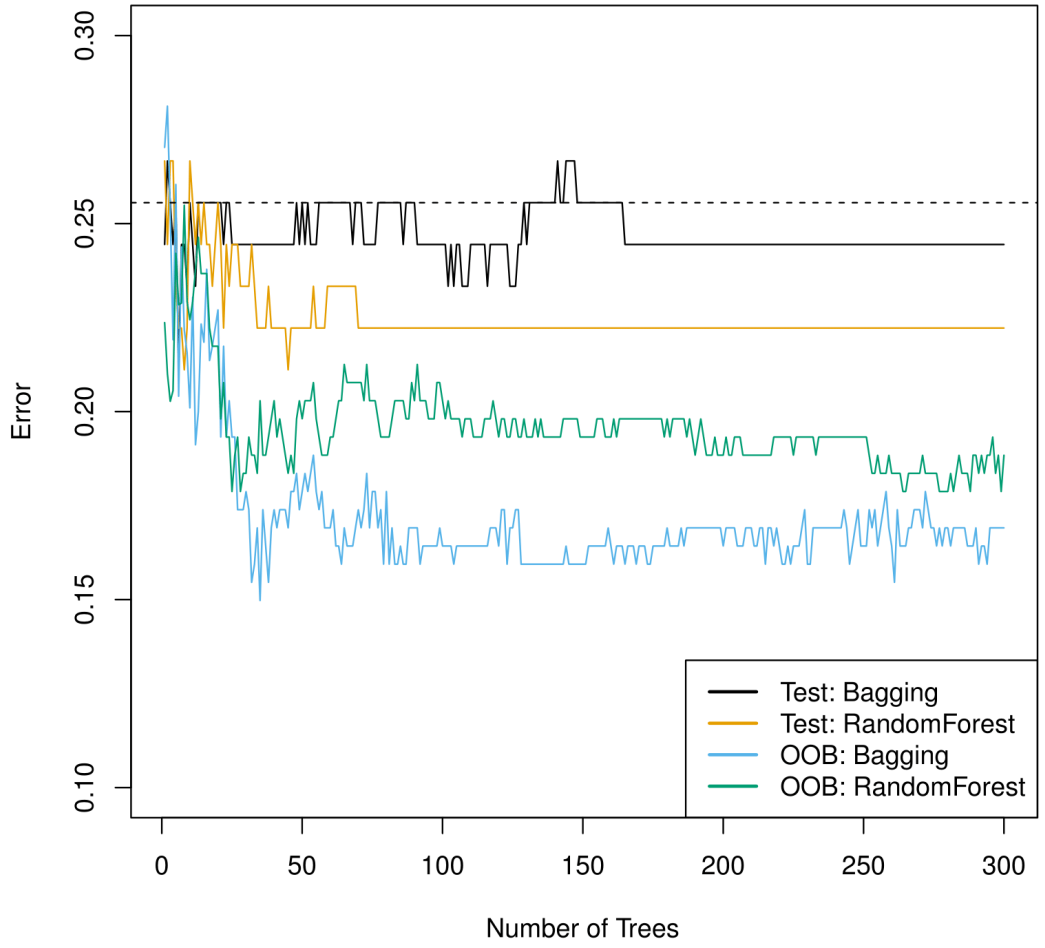
Bagging

Out-of-bag error estimation

- With bagging, we can estimate test error without CV
- On average, each tree uses 2/3 of the observations.
- The remaining 1/3 is out-of-bag (OOB)
- For each sample,
  - obtain prediction only from trees for which the sample was OOB
- Overall error calculated using all samples
- If B is big enough, error is virtually equivalent to LOOCV

Bagging

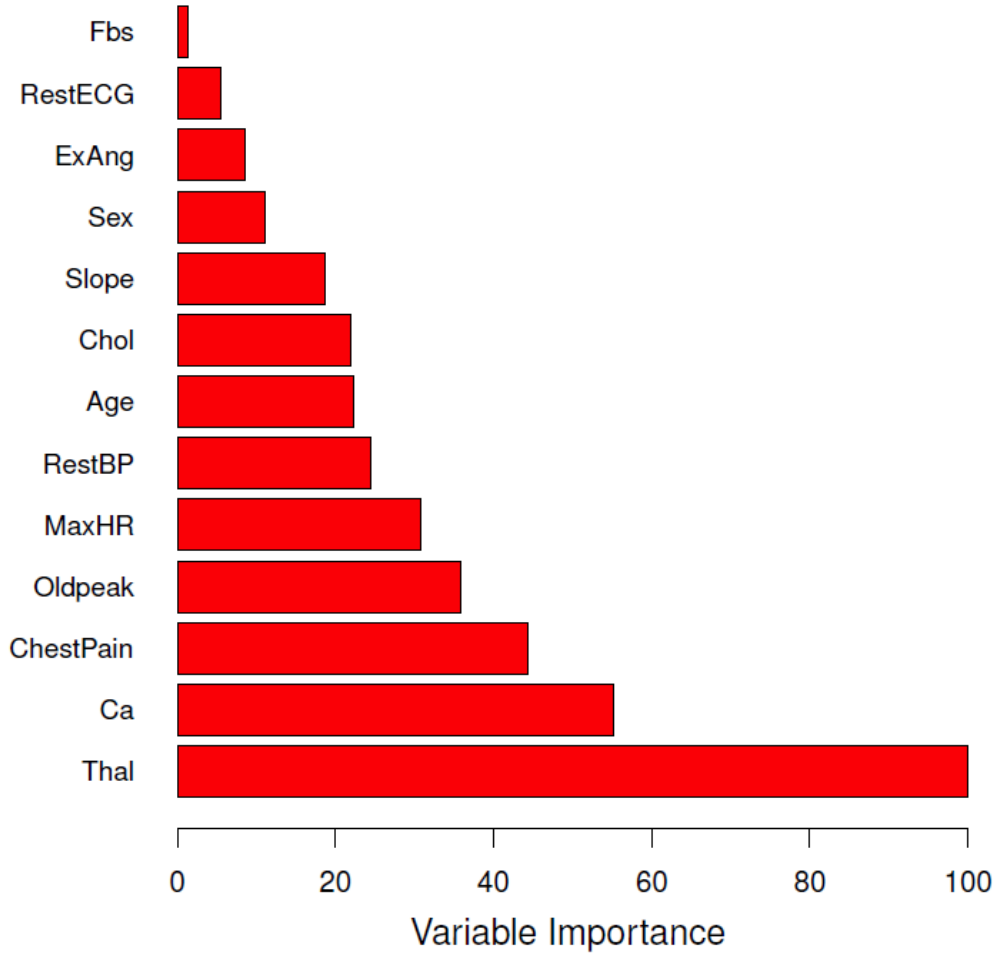
Out-of-bag error estimation



Bagging

Variable Importance Measures

- Interpretability is lost with bagging
- We can rely on variable importance
  - In regression, how much RSS is decreased due to variable splits averaged across the B trees
  - In classification, same with Gini index



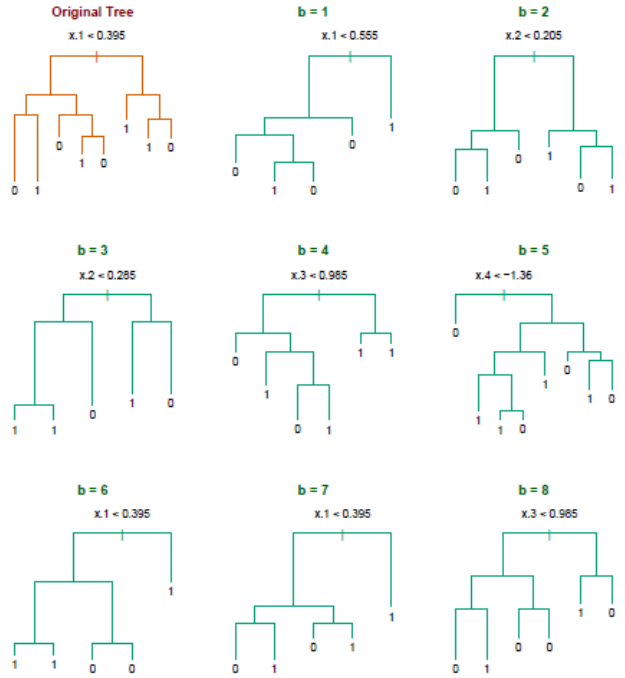
Outline

- Basics of decision trees
  - How to build a tree
  - Tree pruning
  - Classification trees
  - Trees vs linear models
  - Advantages & disadvantages of trees
- Bagging
- **Random forests**
- Boosting

Random forests

Motivation

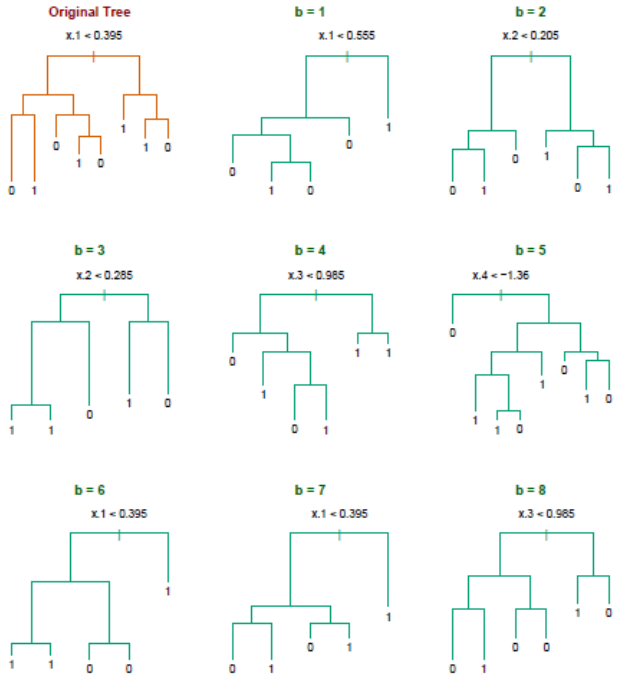
- Bagging can suffer from highly correlated trees
- Example: highly predictive feature is often on top
- We want to decorrelate the trees, to lower the variance of the final prediction.



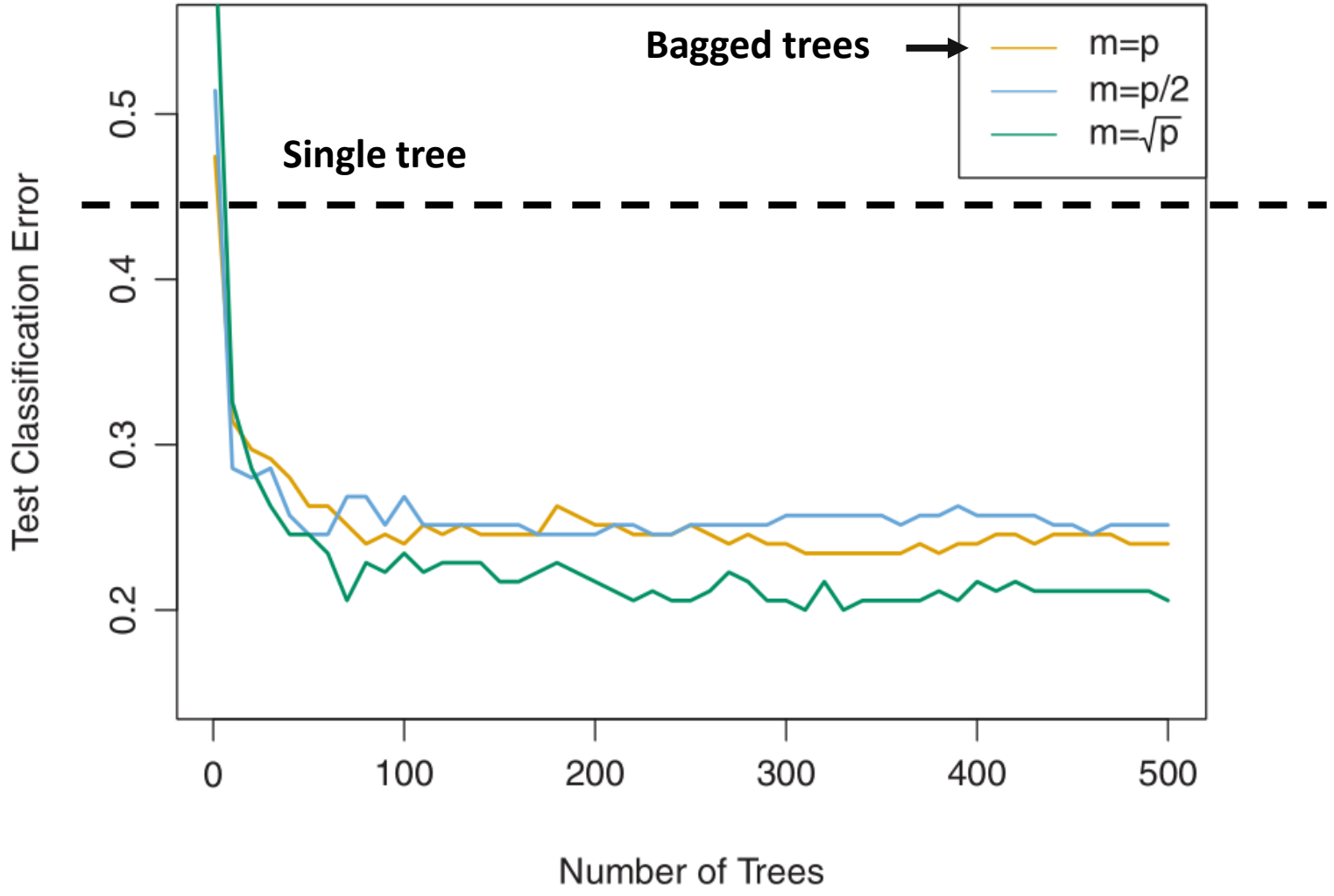
Random forests

Solution

- Suppose there are p total predictors
- At each split, consider only a random subset m of predictors (typically, $m = \sqrt{p}$)
- Resulting decision trees are less correlated
- As a result, they are more robust
- These bagged trees are called *random forests*



Random forests



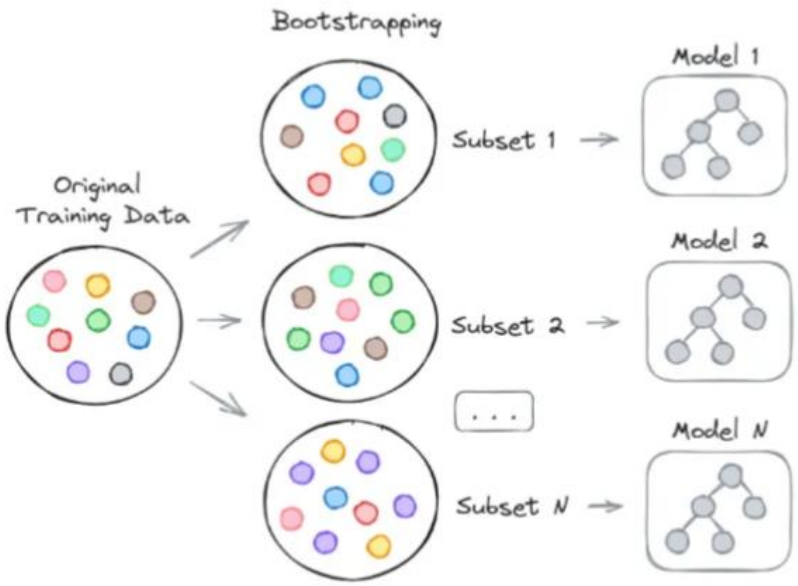
Outline

- Basics of decision trees
  - How to build a tree
  - Tree pruning
  - Classification trees
  - Trees vs linear models
  - Advantages & disadvantages of trees
- Bagging
- Random forests
- **Boosting**

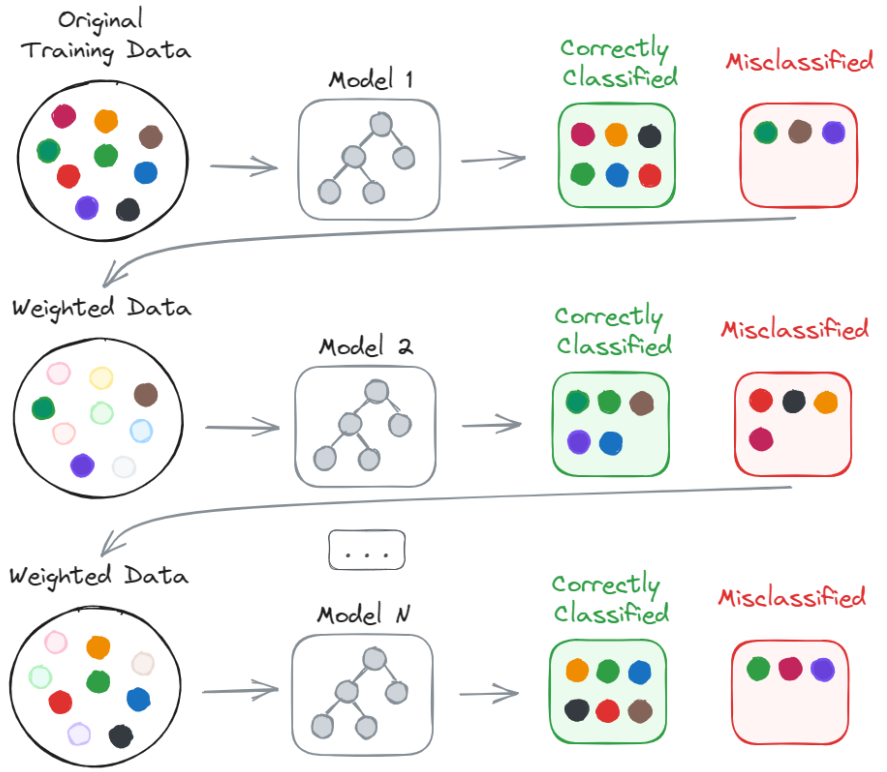
Boosting

- Bagging: fit the same target multiple times
  1. create different bootstraps of the data
  2. fit on the bootstrapped data
  3. bag the predictions
- Boosting: fit prediction errors sequentially
  1. fit the data
  2. fit the errors of the prediction
  3. fit the errors of the errors of the prediction
  4. ...

Boosting vs Bagging

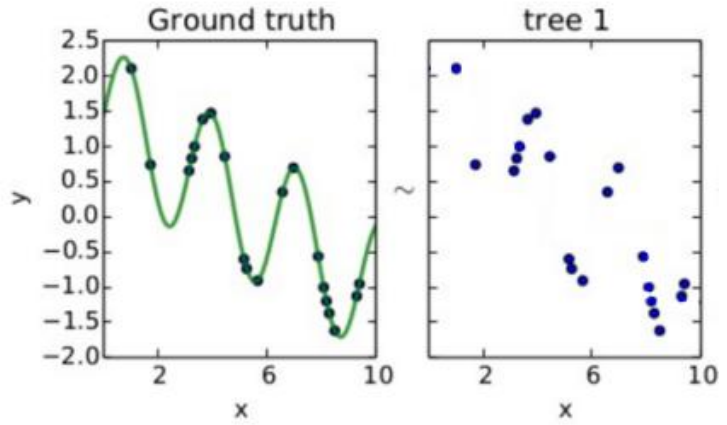


Bagging
(parallel)

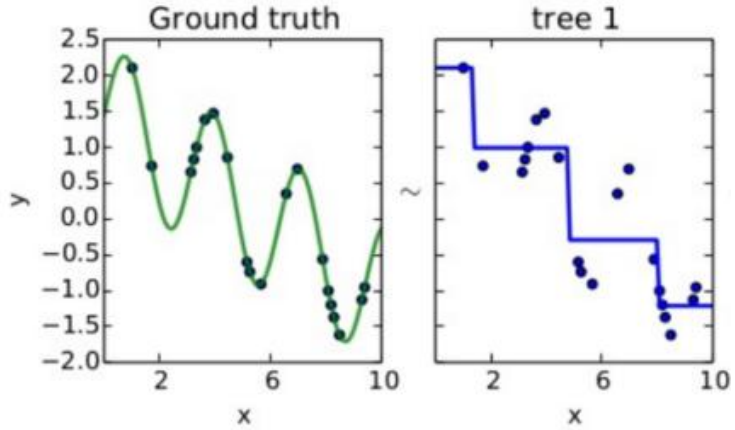


Boosting
(sequential)

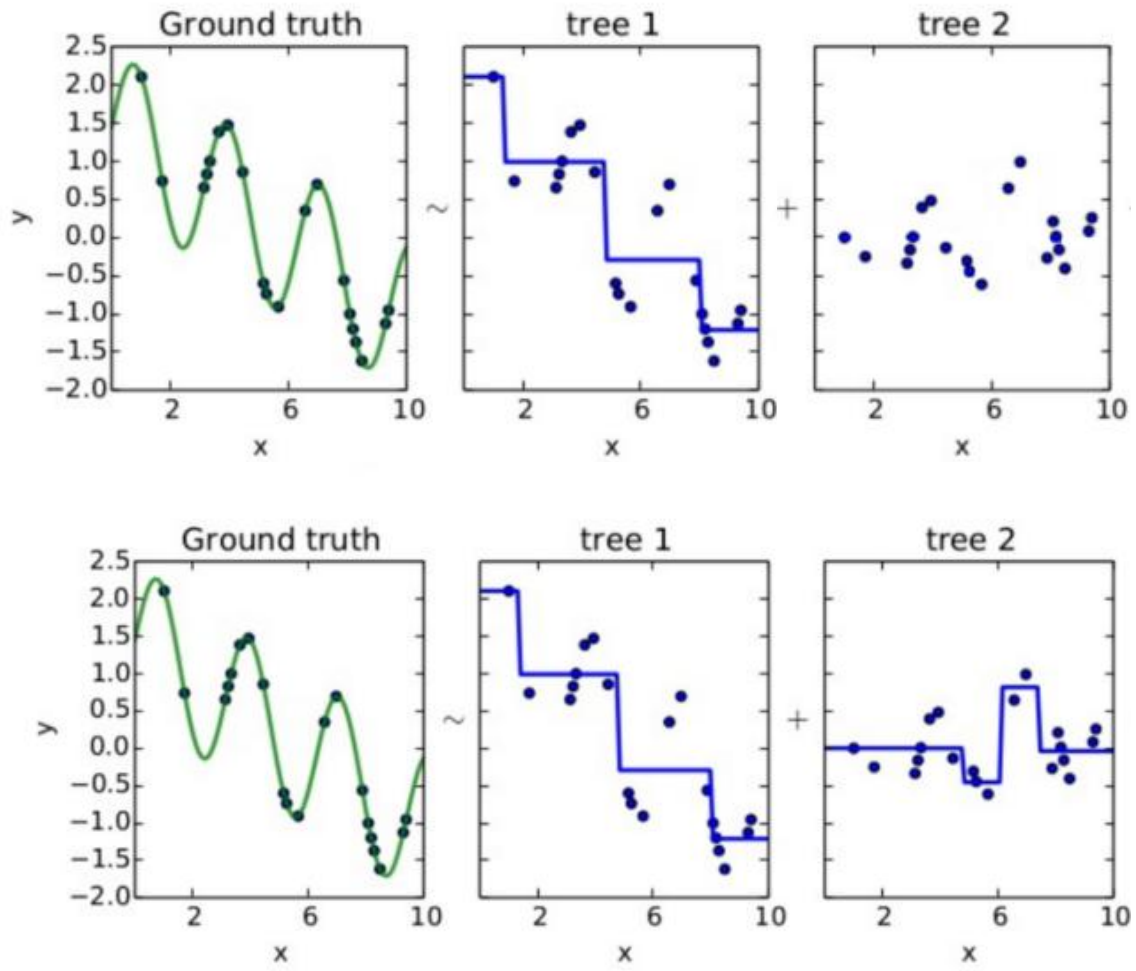
Boosting



Step 1:
fit the data

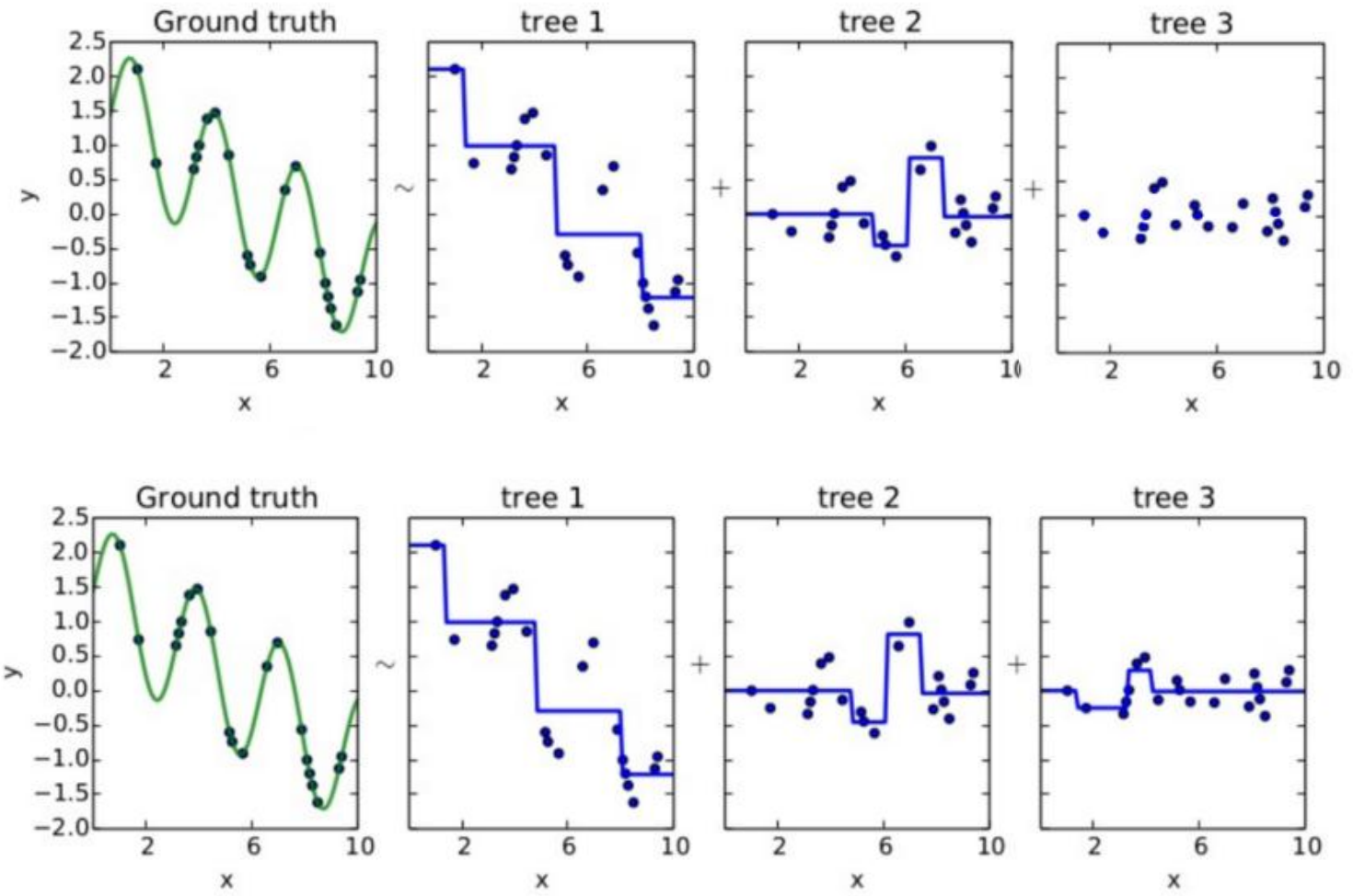


Boosting



Step 2: fit the residuals

Boosting



Step 3:
fit the residuals of the residuals

Boosting

Intuition

- A single large decision tree *fits the data hard* (potentially overfits)
- Boosting instead *learns slowly*
- Each tree should be rather small
- Slowly improve $\hat{f}$ in areas where it does not perform well
- Shrinkage parameter λ slows the process even further
- Classification trees similar but slightly more complex

Boosting

Hyperparameters

1. Number of trees B.
   - Overfitting can occur for B too large. Use CV to select B.
2. Shrinkage parameter λ (learning rate).
   - Typical values are 0.01 or 0.001. Lower λ, higher B and vice versa.
3. Number of splits d in each tree aka *interaction depth*.
   - Often d close to 1 (*stump)*

Boosting

Algorithm

Algorithm 8.2 *Boosting for Regression Trees*

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in the training set.

2. For $b = 1, 2, \ldots, B$, repeat:

   (a) Fit a tree $\hat{f}^b$ with d splits ($d+1$ terminal nodes) to the training data (X, r).

   (b) Update $\hat{f}$ by adding in a shrunken version of the new tree:

   $$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \tag{8.10}$$

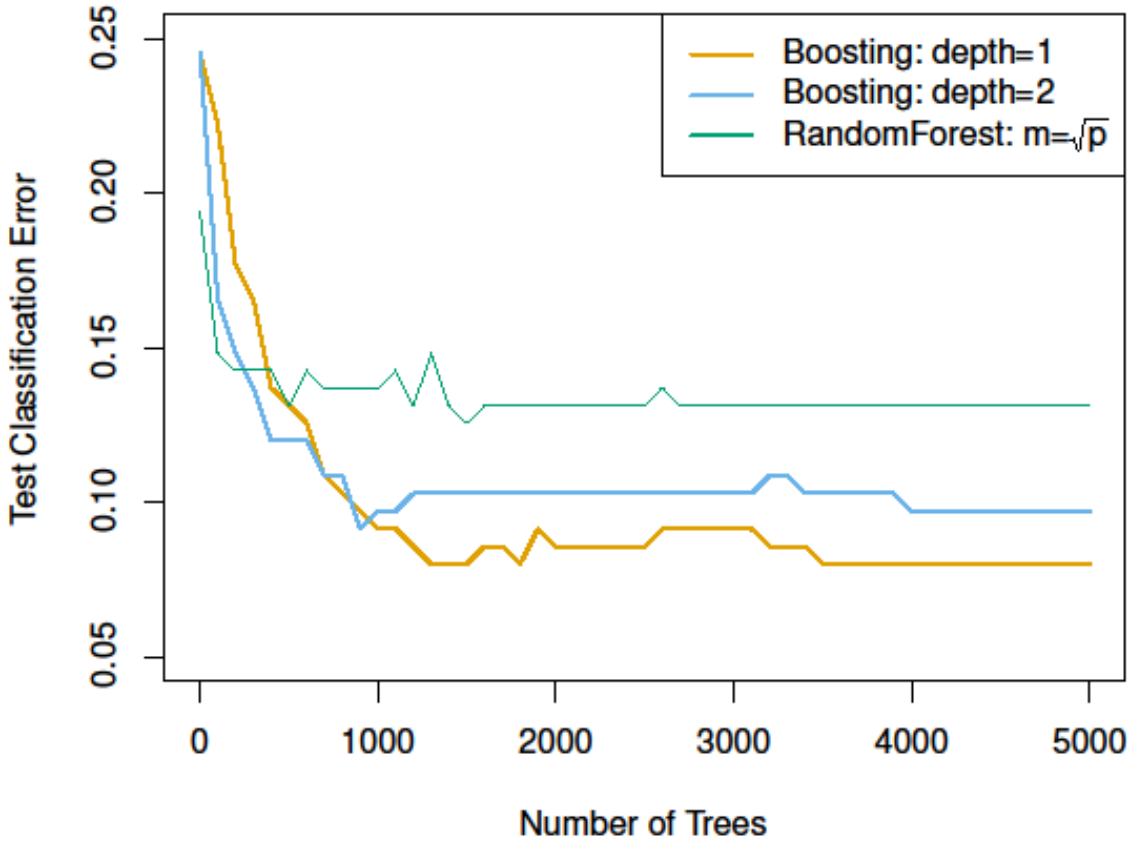
   (c) Update the residuals,

   $$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i). \tag{8.11}$$

3. Output the boosted model,

$$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x). \tag{8.12}$$

Boosting


Boosting: depth=1
Boosting: depth=2
RandomForest: m=√p
Test Classification Error
0.05
0.10
0.15
0.20
0.25
0
1000
2000
3000
4000
5000
Number of Trees

Support vector machines

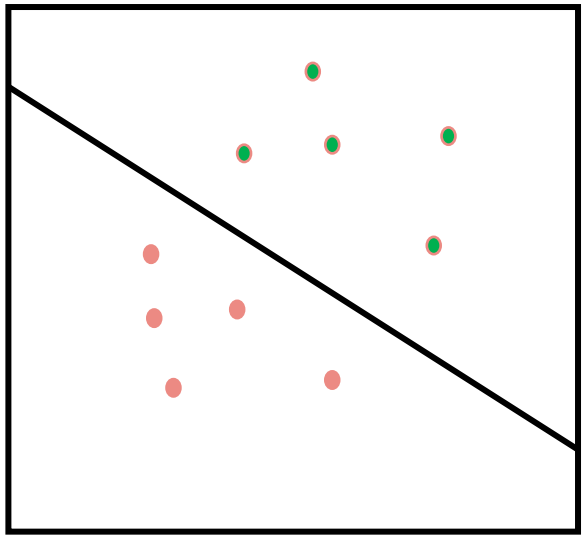
Outline

- **What is a hyperplane?**
- Maximal margin classifier
- Support vector classifier
- Support vector machines
- SVM with more than 2 classes

What is a hyperplane?

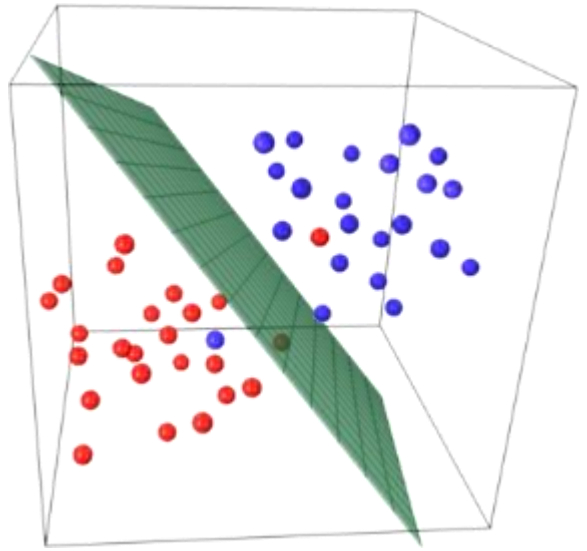
- Consider a *p*-dimensional space
- Hyperplane: flat affine of dimension *p-1*
- Defined by equation: $\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p = 0$
- A *p*-1 dimension hyperplane splits p-dimension in 2

$p = 2$



$1 + 2X_1 + 3X_2 = 0$

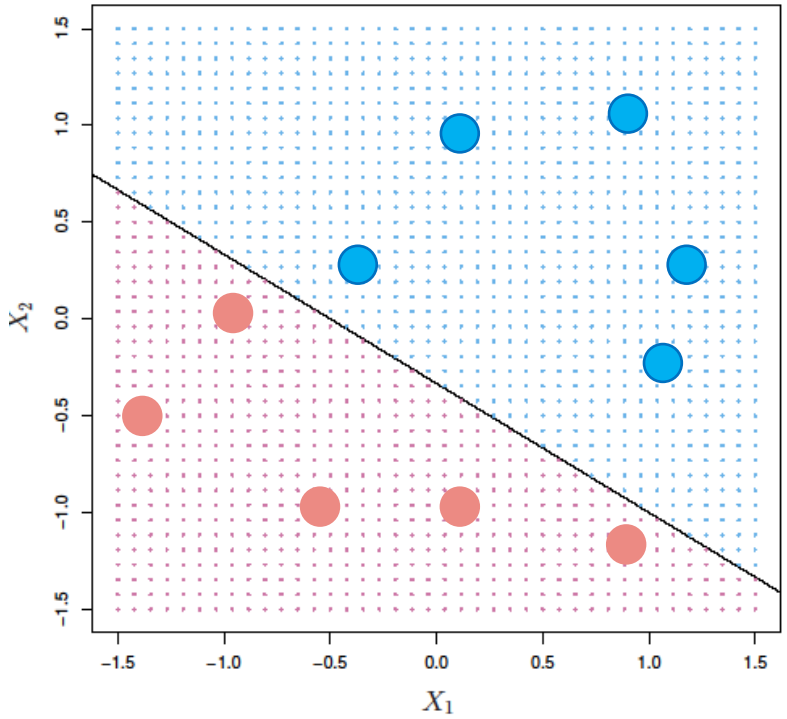
$p = 3$



$3 + 5X_1 + 2X_2 + 1X_3 = 0$

What is a hyperplane?

What is a separating hyperplane?



Equation for hyperplane:

$$1 + 2X_1 + 3X_2 = 0$$

point X^* not on hyperplane:

$1 + 2X_1^* + 3X_2^* < 0$ → red class

$1 + 2X_1^* + 3X_2^* > 0$ → blue class

A hyperplane that separates two classes.

What is a hyperplane?

Classification using a separating hyperplane

- Suppose we have a $n \times p$ data matrix $\boldsymbol{X}$
- These n points live in p-dimensional space

$$x_1 = \begin{pmatrix} x_{11} \\ \vdots \\ x_{1p} \end{pmatrix}, \ldots, x_n = \begin{pmatrix} x_{n1} \\ \vdots \\ x_{np} \end{pmatrix}$$

- Classes $y_1, \ldots, y_n \in \{-1,1\}$
- How to classify test point $x^* = \begin{pmatrix} x_1^* & \ldots & x_p^* \end{pmatrix}^T$?

Use the separating hyperplane!

What is a hyperplane?

Classification using a separating hyperplane

- Above / below hyperplane determines class
- $p-1$ dim. hyperplane should obey for all X_i:

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip} > 0 \text{ if } y_i = 1$$

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip} < 0 \text{ if } y_i = -1$$

- Or in one equation:

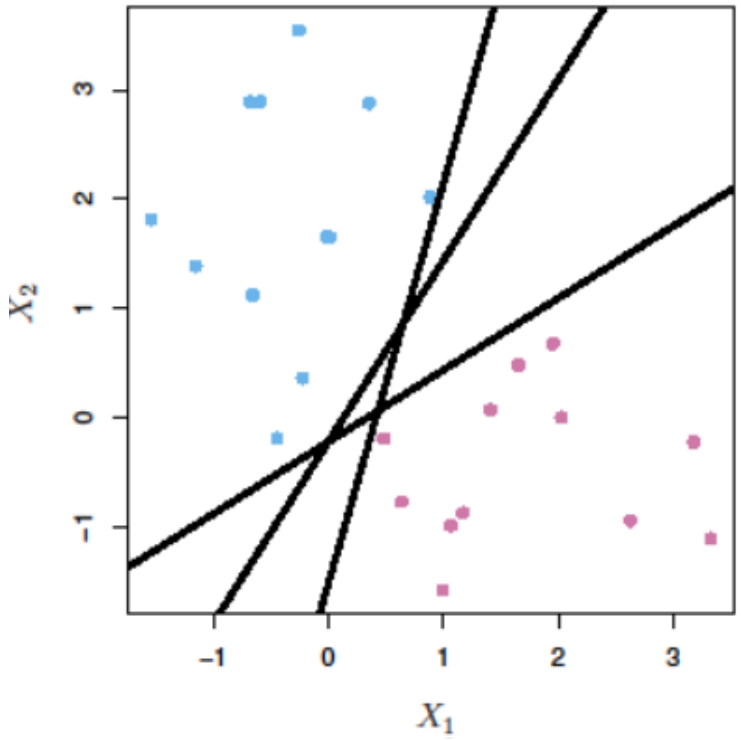
$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip}) > 0$$

- We need to find $\beta_i' s$ that obey this equation.

What is a hyperplane?

Classification using a separating hyperplane

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip}) > 0$$



But, which one?

Outline

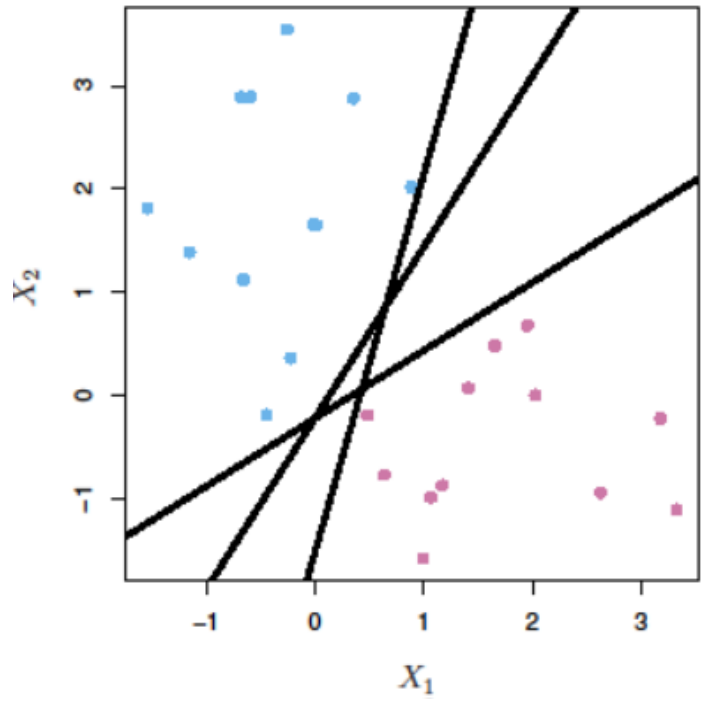
- What is a hyperplane?
- **Maximal margin classifier**
- Support vector classifier
- Support vector machines
- SVM with more than 2 classes

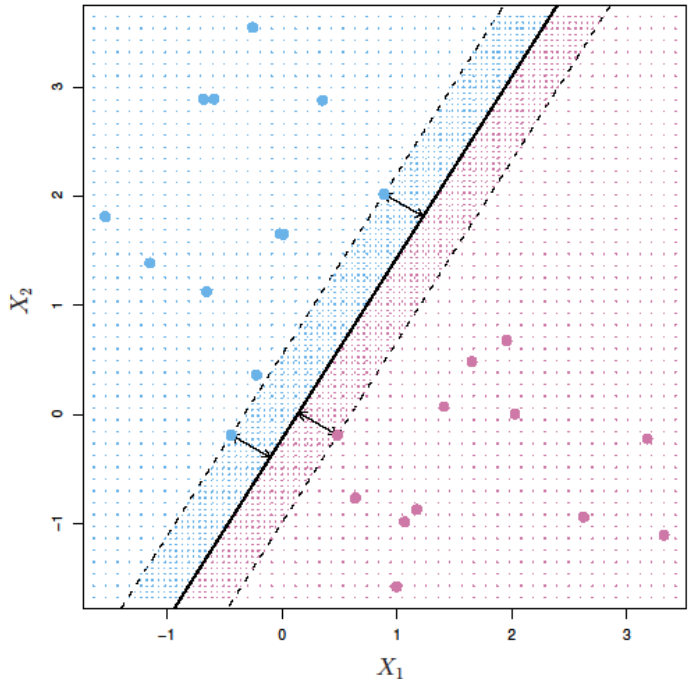
What is a hyperplane?

Intuition

Maximal margin classifier

- Which hyperplane to choose?
- Maximise the margin (distance to closest point)!
- *Maximal margin (optimal separating) hyperplane*





Maximal margin classifier

Maximal margin hyperplane is solution to this optimisation problem

$$\underset{\beta_0,\beta_1,\ldots,\beta_p,M}{\text{maximize}} \; M$$

margin

$$\text{subject to} \sum_{j=1}^{p} \beta_j^2 = 1,$$

needed to interpret as distance

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip}) \geq M \;\; \forall\, i = 1, \ldots, n$$

perpendicular distance of point to separating hyperplane

Maximal margin classifier

Properties

- Perpendicular distance of point i to the hyperplane is:

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip})$$

- Therefore, each point in $\boldsymbol{X}$ is at least a distance M from the separating hyperplane:

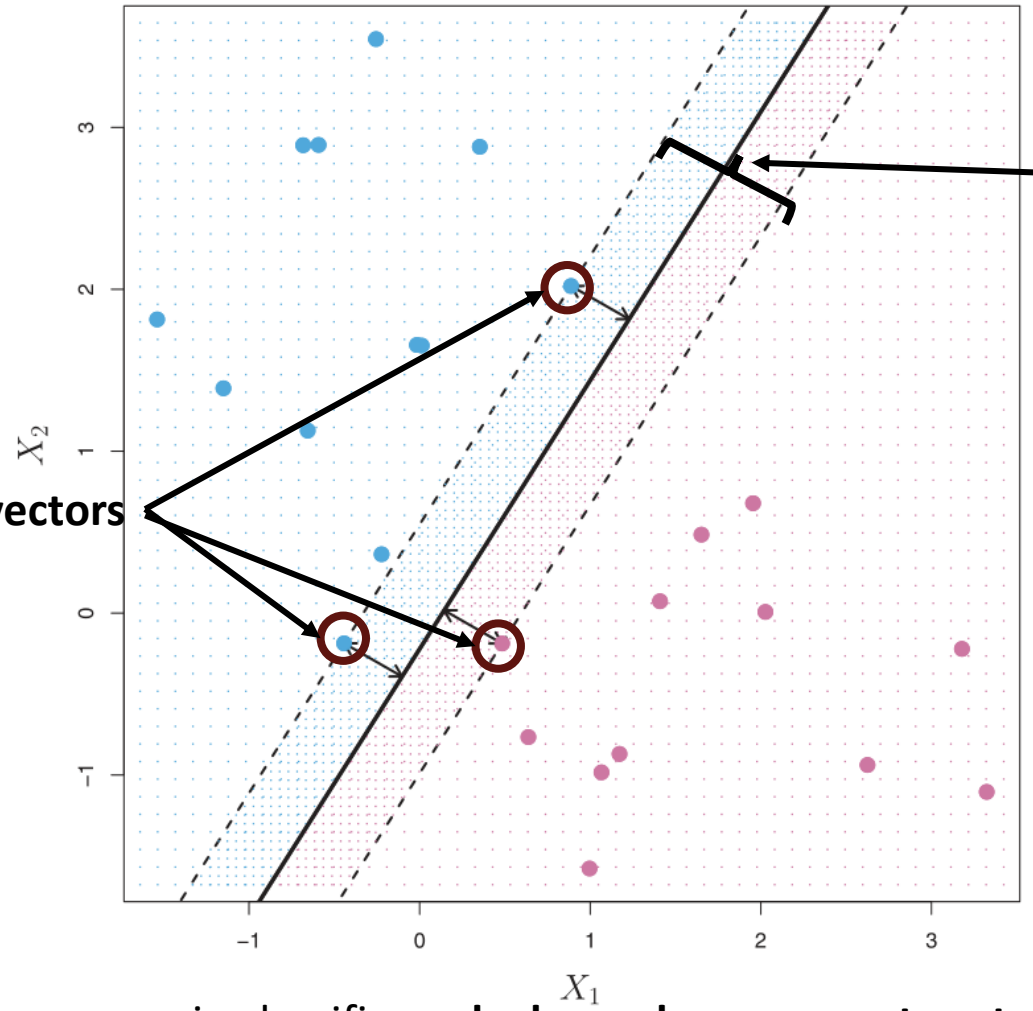
$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip}) \geq M \;\; \forall\, i = 1, \ldots, n$$

Maximal margin classifier

Properties

- We hope that large margin on training data → large margin on test data
- We can use distance to hyperplane as classification certainty
- MMC might lead to overfitting when p is high

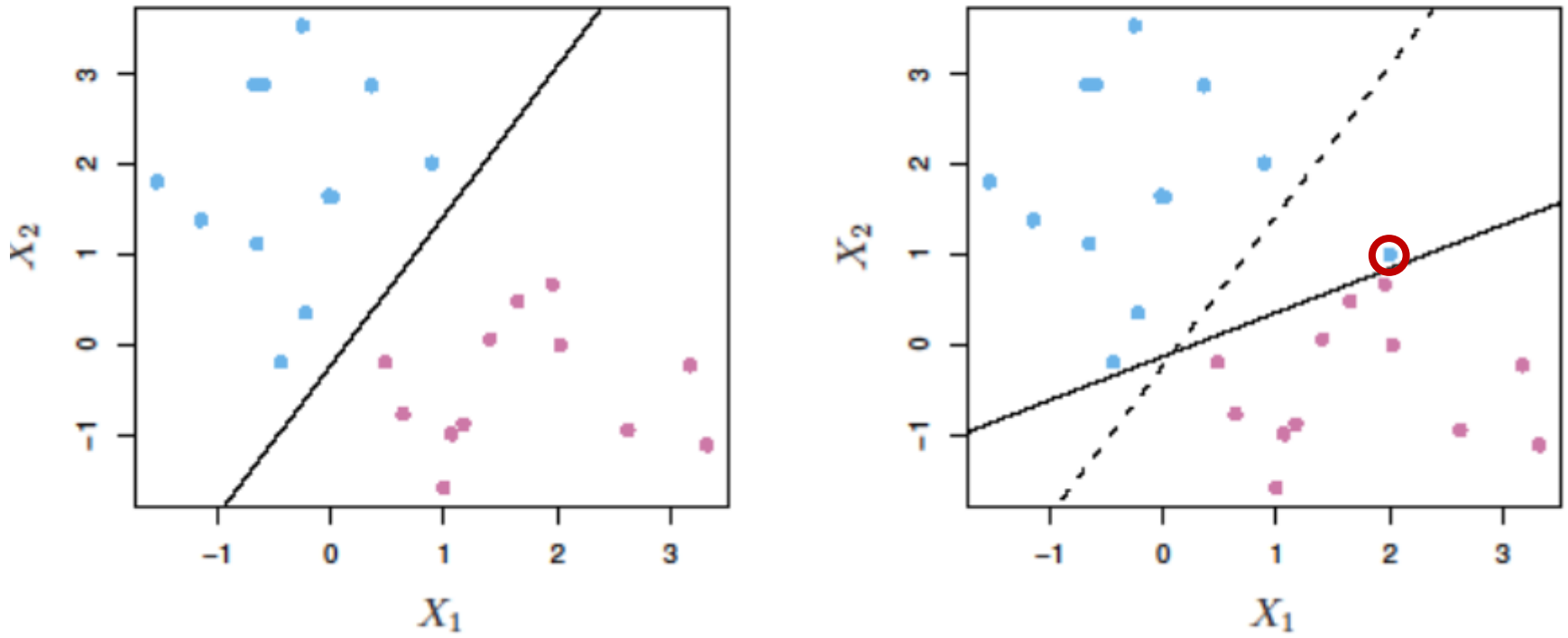
Maximal margin classifier



The maximum margin classifier **only depends on support vectors**!

Maximal margin classifier

Maximal margin classifier is not very robust.
Associated with overfitting

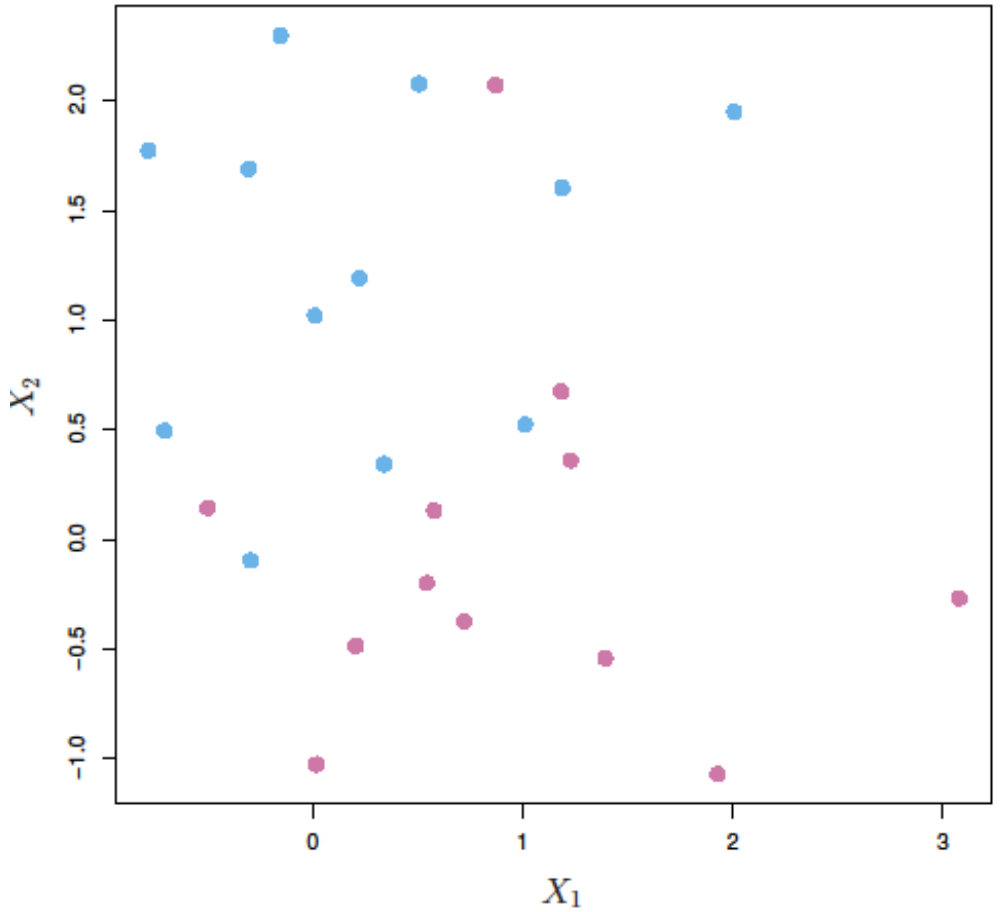


Outline

- What is a hyperplane?
- Maximal margin classifier
- **Support vector classifier**
- Support vector machines
- SVM with more than 2 classes

Support vector classifier

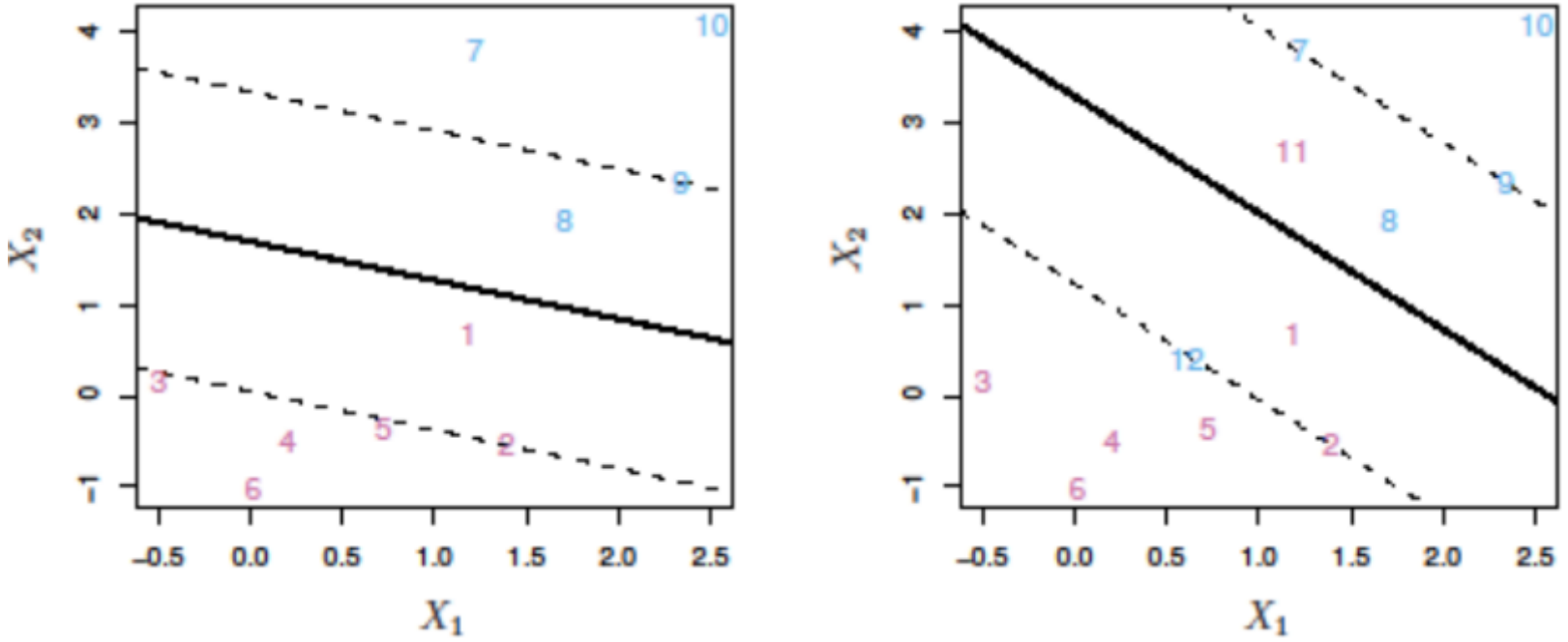
Problem: non-separable data



Let's cut ourselves some slack

Support vector classifier

- SVC is a generalisation of the MMC
- It allows for errors in the classifier (soft margins)
  - Points within the margin
  - Points on the wrong side of the hyperplane



Support vector classifier

Solution: Add slack variables to MMC optimisation problem

$$\underset{\beta_0,\beta_1,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n,M}{\text{maximize}} \quad M$$

$$\text{subject to} \quad \sum_{j=1}^{p} \beta_j^2 = 1,$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip}) \geq M(1 - \epsilon_i)$$

$$\epsilon_i \geq 0, \quad \sum_{i=1}^{n} \epsilon_i \leq C,$$

slack variables

budget

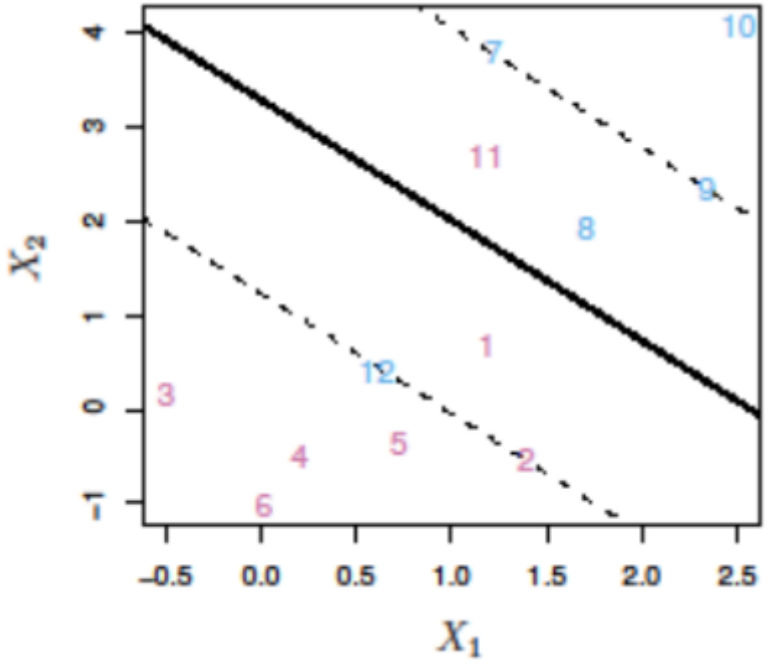
Support vector classifier

Slack variables

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip}) \geq M(1 - \epsilon_i)$$

$$\epsilon_i \geq 0, \quad \sum_{i=1}^{n} \epsilon_i \leq C,$$

- If $0 < \epsilon_i < 1$:
  wrong side of margin (8, 1)
- If $\epsilon_i > 1$:
  wrong side of hyperplane
  (11, 12)
- Total error $\leq C$

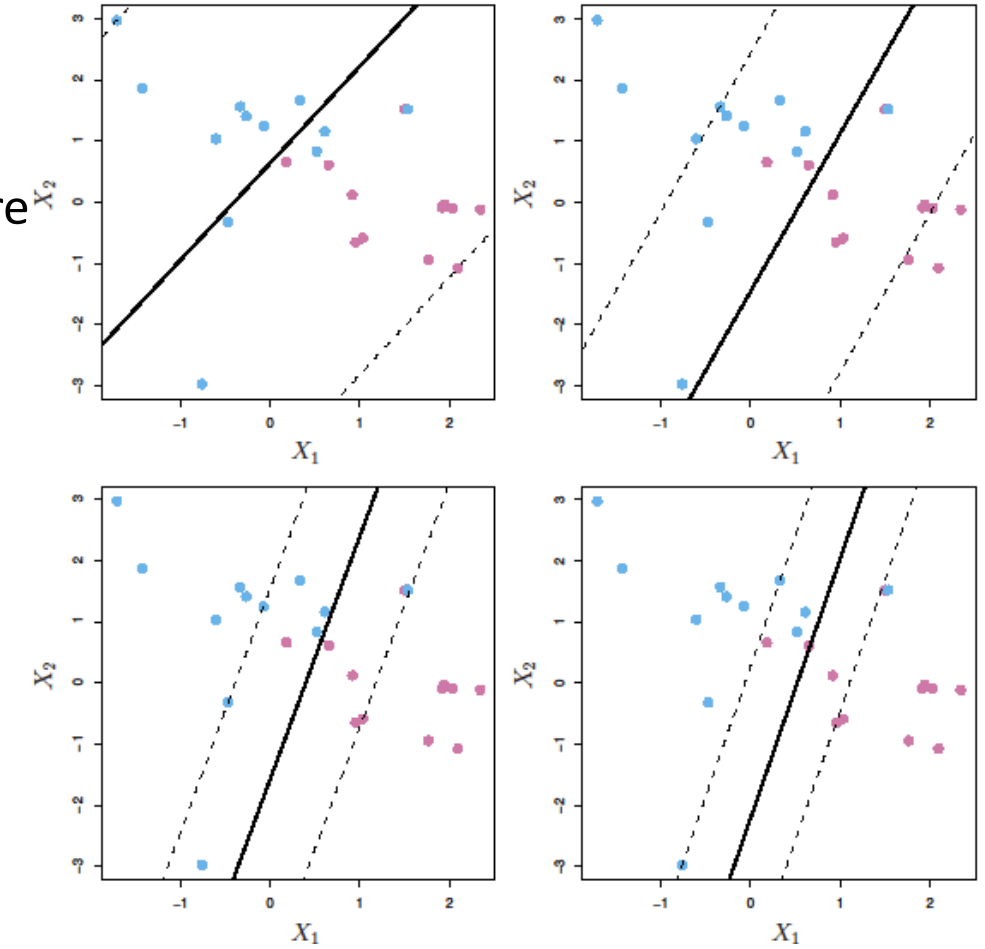


Support vector classifier

C controls the bias-variance trade-off

C high

- Wide margins
- We allow more violations
- Fit data less hard
- High bias
- Low variance



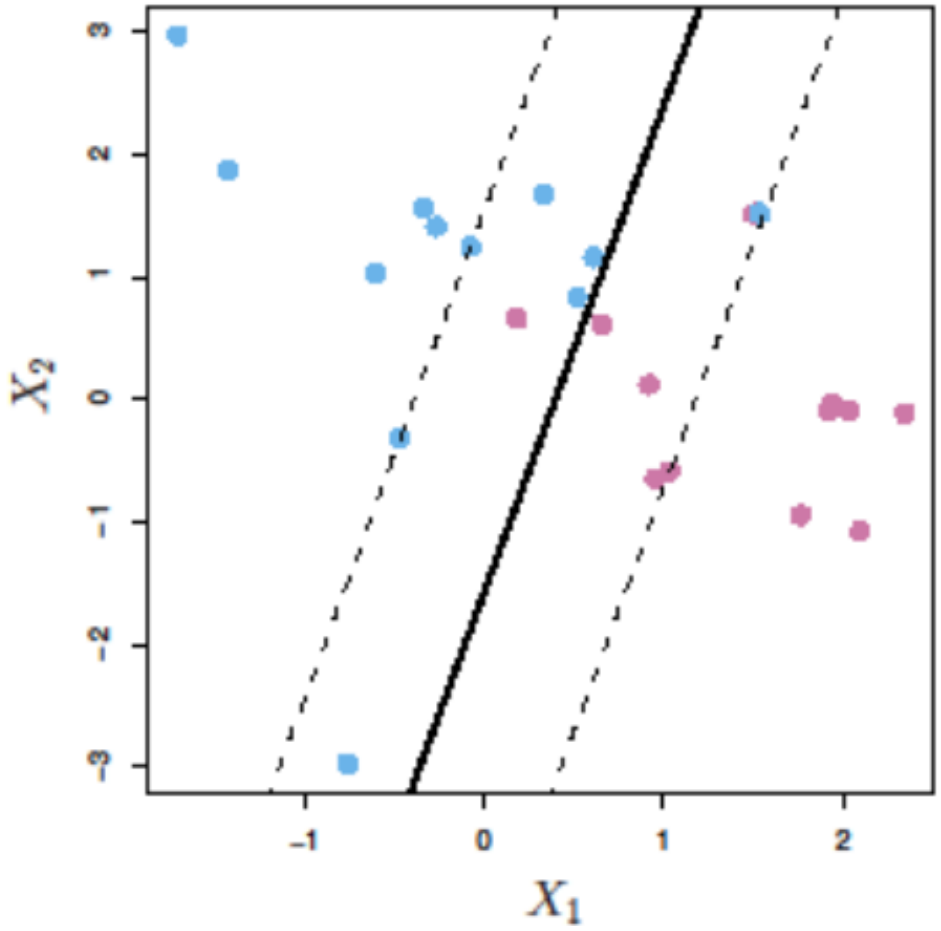
C low

- Narrow margins
- Rarely violated
- Highly fit to data
- Low bias
- High variance

Support vector classifier

Support vectors

- On margin
- Within margin
- Wrong side of the hyperplane



Everything on or between the dotted lines

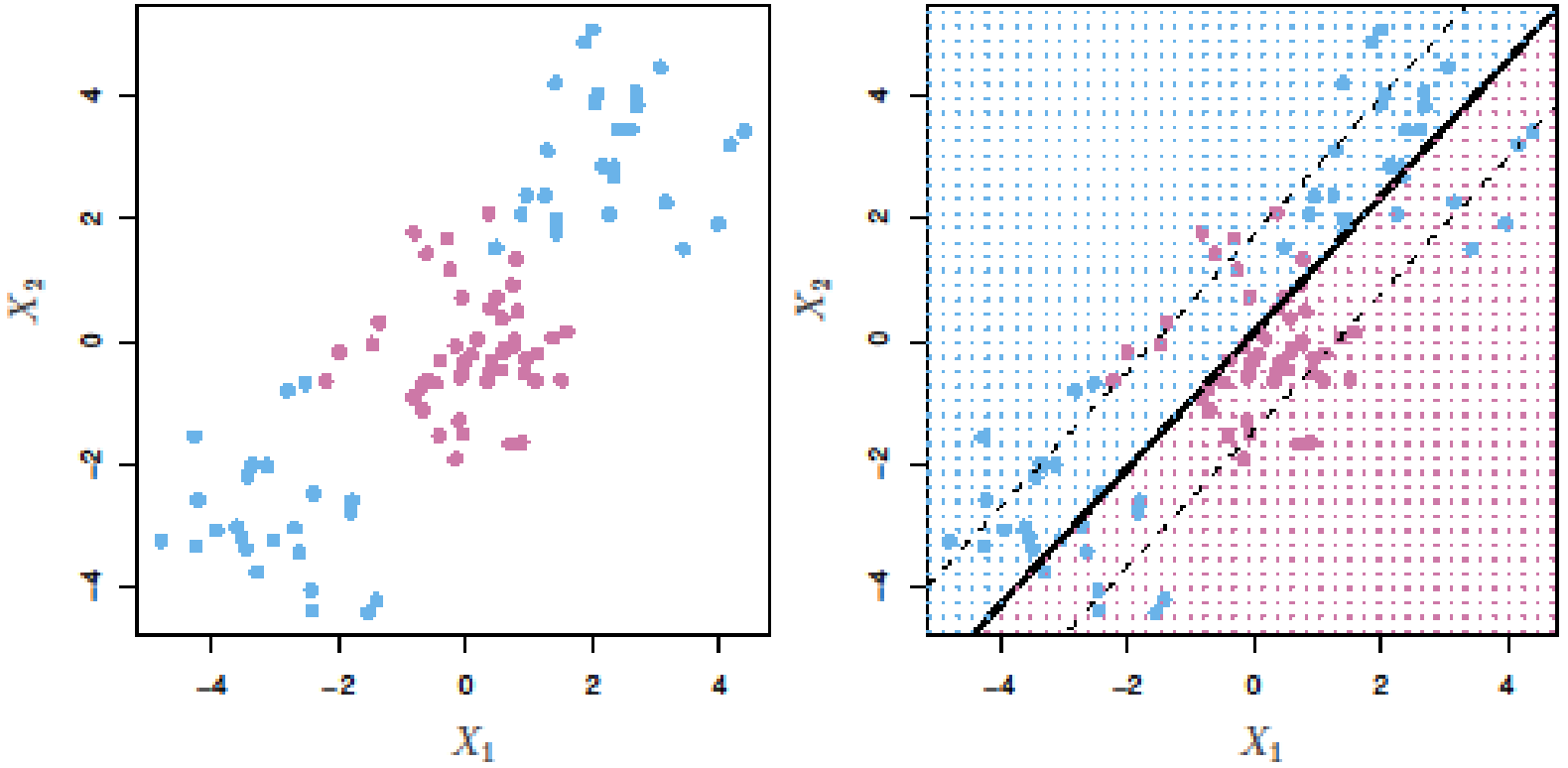
Only support vectors affect the classifier

Outline

- What is a hyperplane?
- Maximal margin classifier
- Support vector classifier
- **Support vector machines**
- SVM with more than 2 classes

Support vector machines

Motivation



We need non-linear decision boundaries!

Support vector machines

Non-linear decision boundaries

Standard way to create non-linear decision boundary: transform variables

$$X_1, X_2, \ldots, X_p \quad \longrightarrow \quad X_1, X_1^2, X_2, X_2^2, \ldots, X_p, X_p^2$$

$$\underset{\beta_0, \beta_{11}, \beta_{12} \ldots, \beta_{p1}, \beta_{p2}, \epsilon_1, \ldots, \epsilon_n, M}{\text{maximize}} M$$

$$\text{subject to } y_i \left(\beta_0 + \sum_{j=1}^{p} \beta_{j1} x_{ij} + \sum_{j=1}^{p} \beta_{j2} x_{ij}^2 \right) \geq M(1 - \epsilon_i)$$

$$\sum_{i=1}^{n} \epsilon_i \leq C, \quad \epsilon_i \geq 0, \quad \sum_{j=1}^{p} \sum_{k=1}^{2} \beta_{jk}^2 = 1.$$

Support vector machines

Non-linear decision boundaries

- Number of possibilities for non-linear transformations is very large
  - Higher order polynomials
  - Interaction terms
- Quickly too many variables in the model: computationally too expensive
- Support Vector Machines use a more efficient method
  - SVMs = SVC with *kernels*

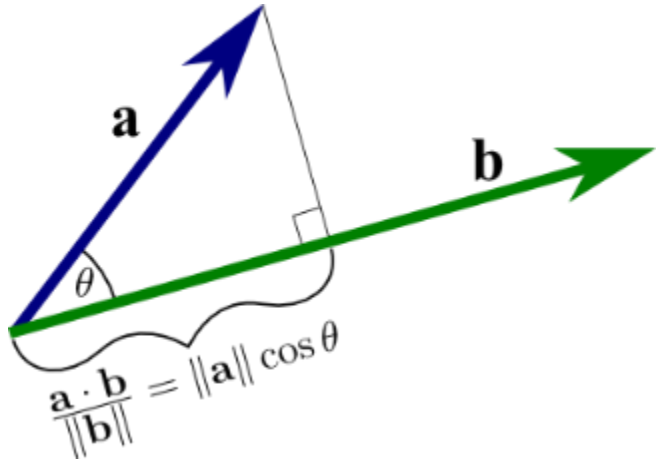
Support vector machines

Inner-products

- The solution to support vector classifier depends **only on the inner products** of the data points.

$$\langle x_i, x_{i'} \rangle = \sum_{j=1}^{p} x_{ij} x_{i'j}.$$

$$\vec{a} \,.\, \vec{b} = \langle \vec{a}\,, \vec{b} \rangle = \sum_{j}^{p} a_j b_j$$



Support vector machines

Inner-products

- The solution to support vector classifier depends **only on the inner products** of the data points.

$$\langle x_i, x_{i'} \rangle = \sum_{j=1}^{p} x_{ij} x_{i'j}.$$

Linear SVC: $$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i \langle x, x_i \rangle$$

new point

parameters

Equivalent to: $$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle x, x_i \rangle$$

support vectors

Support vector machines

Inner-products

- Change form of inner product.

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle x, x_i \rangle$$

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(x, x_i)$$

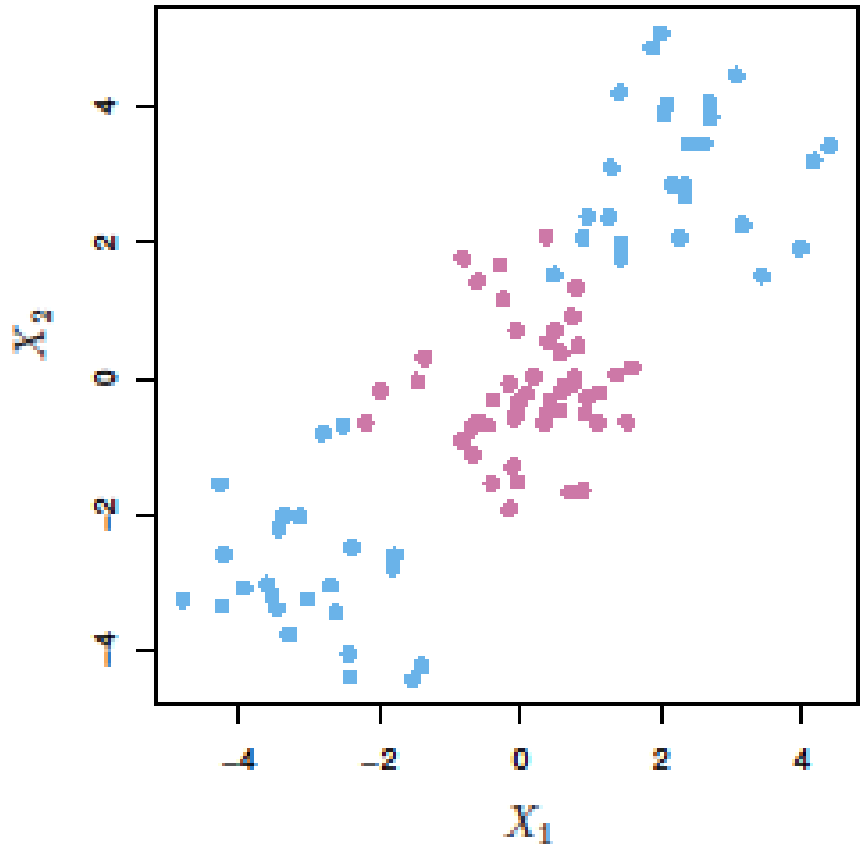
Different choices of *kernel* $K(x, x_i)$ possible:

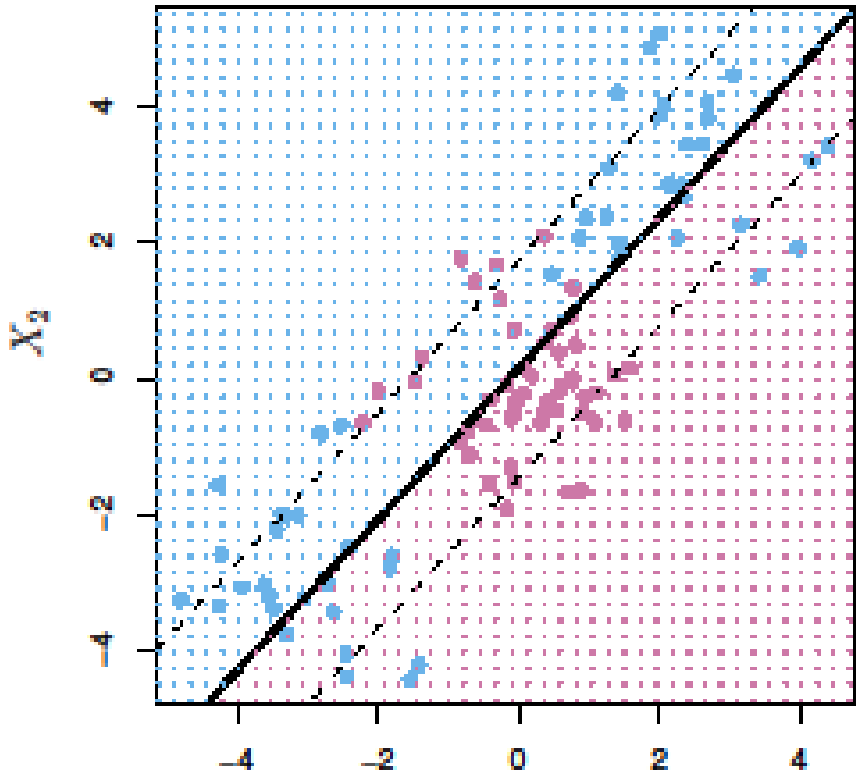
- linear: $K(x_i, x_{i'}) = \sum_{j=1}^{p} x_{ij} x_{i'j}$
- polynomial: $K(x_i, x_{i'}) = (1 + \sum_{j=1}^{p} x_{ij} x_{i'j})^d$
- radial: $K(x_i, x_{i'}) = \exp(-\gamma \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2)$

Support vector machines

Kernels

Example



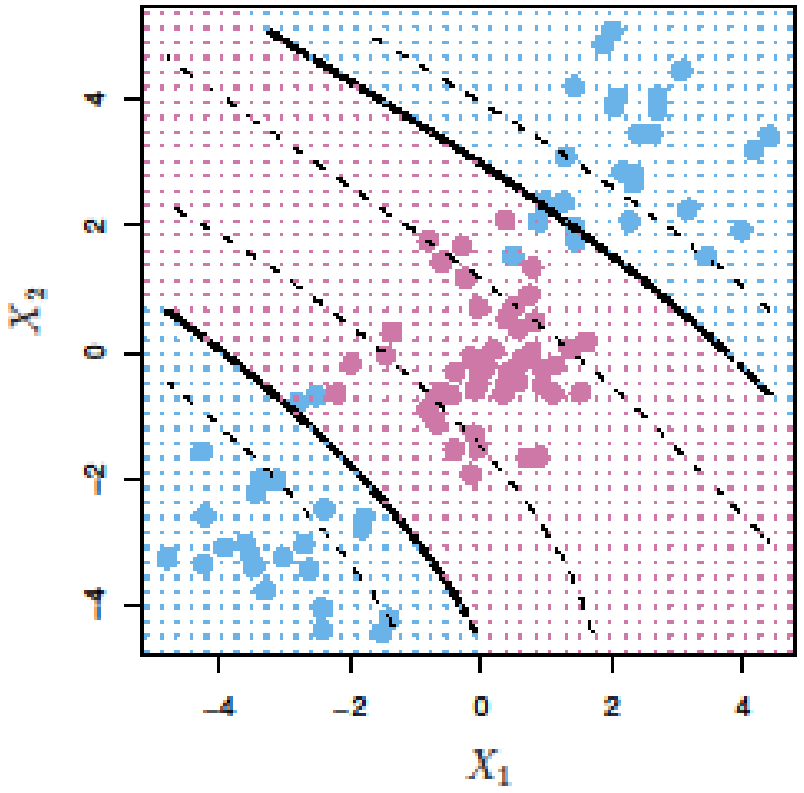


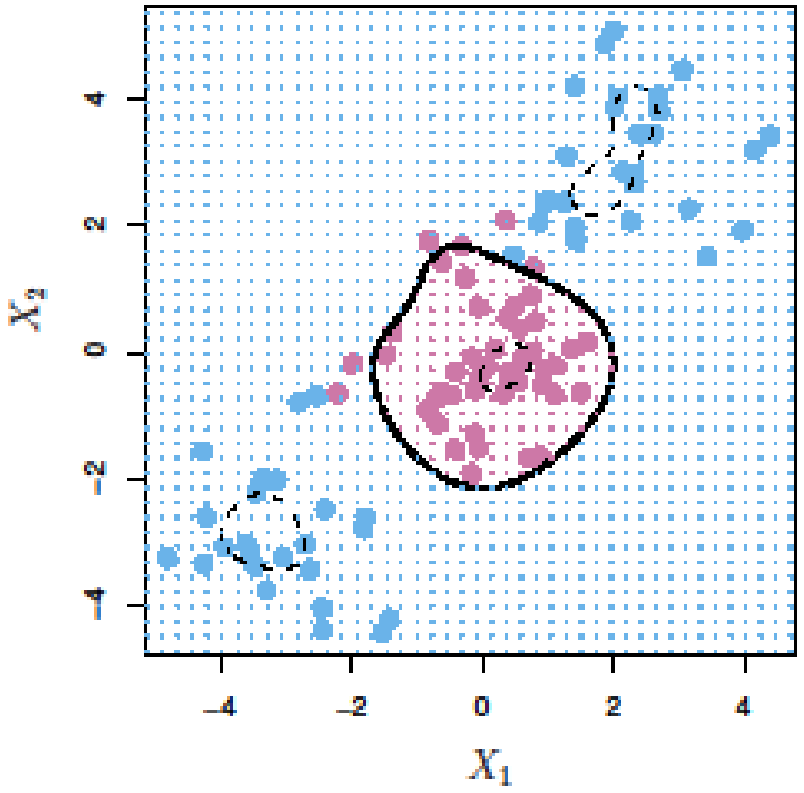
$$K(x_i, x_{i'}) = \sum_{j=1}^{p} x_{ij} x_{i'j}$$

Support vector machines

Kernels

Example





$$K(x_i, x_{i'}) = (1 + \sum_{j=1}^{p} x_{ij} x_{i'j})^d$$

$$K(x_i, x_{i'}) = \exp(-\gamma \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2)$$

Support vector machines

How does the radial kernel work?

$$K(x_i, x_{i'}) = \exp(-\gamma \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2)$$

positive constant

- If test observation x^* is far from training observation x_i, $\sum_{j=1}^{p} (x_j^* - x_{ij})^2$ will be large
- So, $\exp(-\gamma \sum_{j=1}^{p} (x_j^* - x_{ij})^2)$ will be tiny
- Training samples far from x^* will have no effect on its classification
- This means radial kernel is very *local*

Support vector machines

Advantages of using kernels

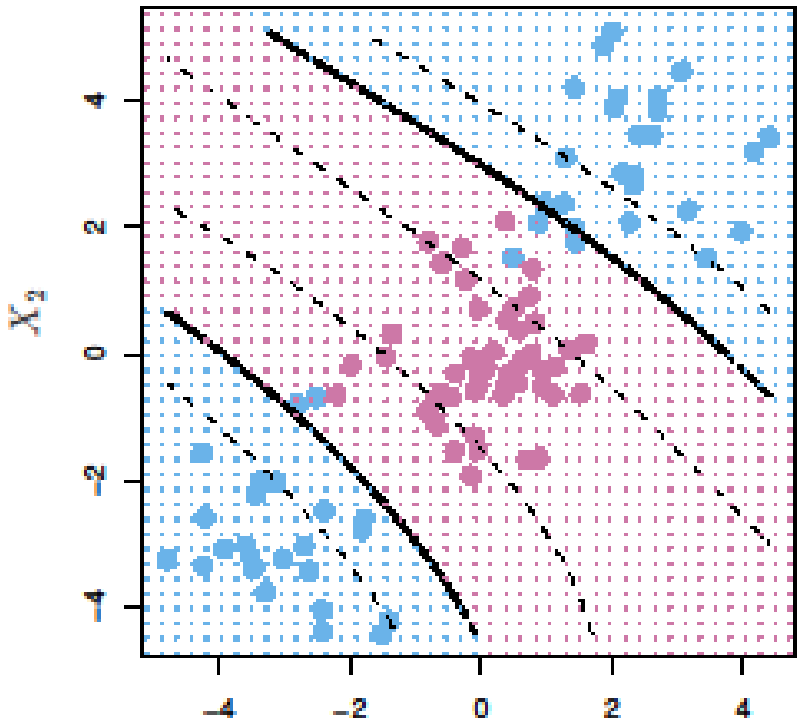
- Kernel trick: choose inner product $K(x, x_i)$
  - This implies new geometry of feature space
- New geometry has different properties
  - should have better separation properties
- Computational efficiency vs enlarging feature space
  - only have to calculate $K(x, x_i)$ for all $\binom{n}{2}$ pairs without working in enlarged feature space
  - In some cases enlarged feature space is infinite dimensional!

Support vector machines

Kernels

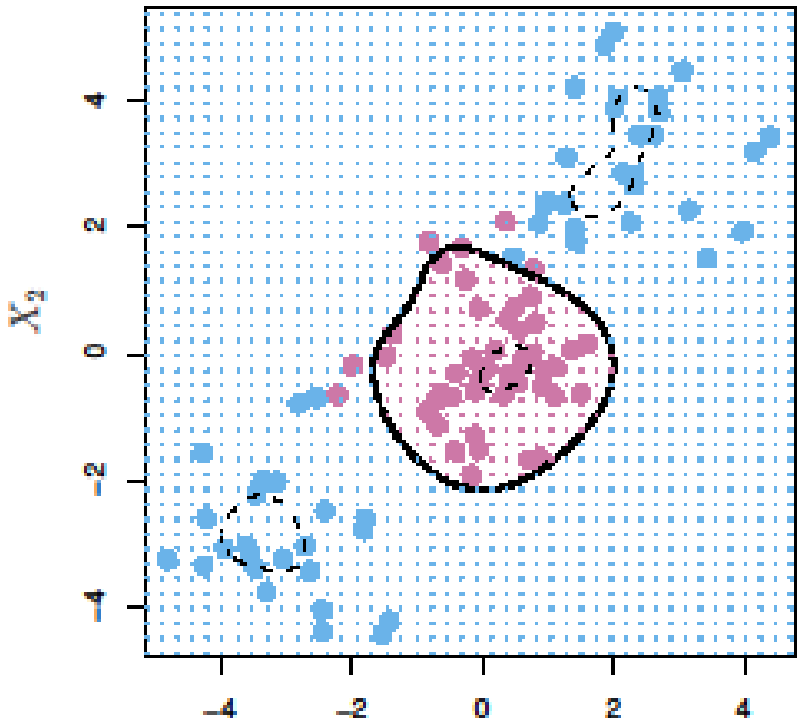
Example

non-linear geometry



$$K(x_i, x_{i'}) = (1 + \sum_{j=1}^{p} x_{ij} x_{i'j})^d$$

non-linear geometry
infinite dimensional



$$K(x_i, x_{i'}) = \exp(-\gamma \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2)$$

Outline

- What is a hyperplane?
- Maximal margin classifier
- Support vector classifier
- Support vector machines
- **SVM with more than 2 classes**

SVM with more than 2 classes

One-Versus-One Classification

- What to do if $K > 2$ classes?
- Option 1:
  - train $\binom{K}{2}$ SVMs for each pair of classes
  - Count how many times a test observation is put into a class
  - Classify it as the one that occurs the most

example: 3 classes R vs G vs B

$\binom{3}{2} = 3$ pairs: R vs G R vs B G vs B

test point: R R B ⟶ R

SVM with more than 2 classes

One-Versus-All Classification

What to do if $K > 2$ classes?

- Option 2:
  - Train K SVMs
  - Each SVM compares one class versus all other classes
  - Assign test observation to SVM with highest confidence (largest distance from separating hyperplane)

example: 3 classes	R vs G vs B			
3 SVMs:	R vs G or B	G vs R or B	B vs R or G	
test point:	R: 0.47	G: 0.21	R or G: 0.52	→ R